

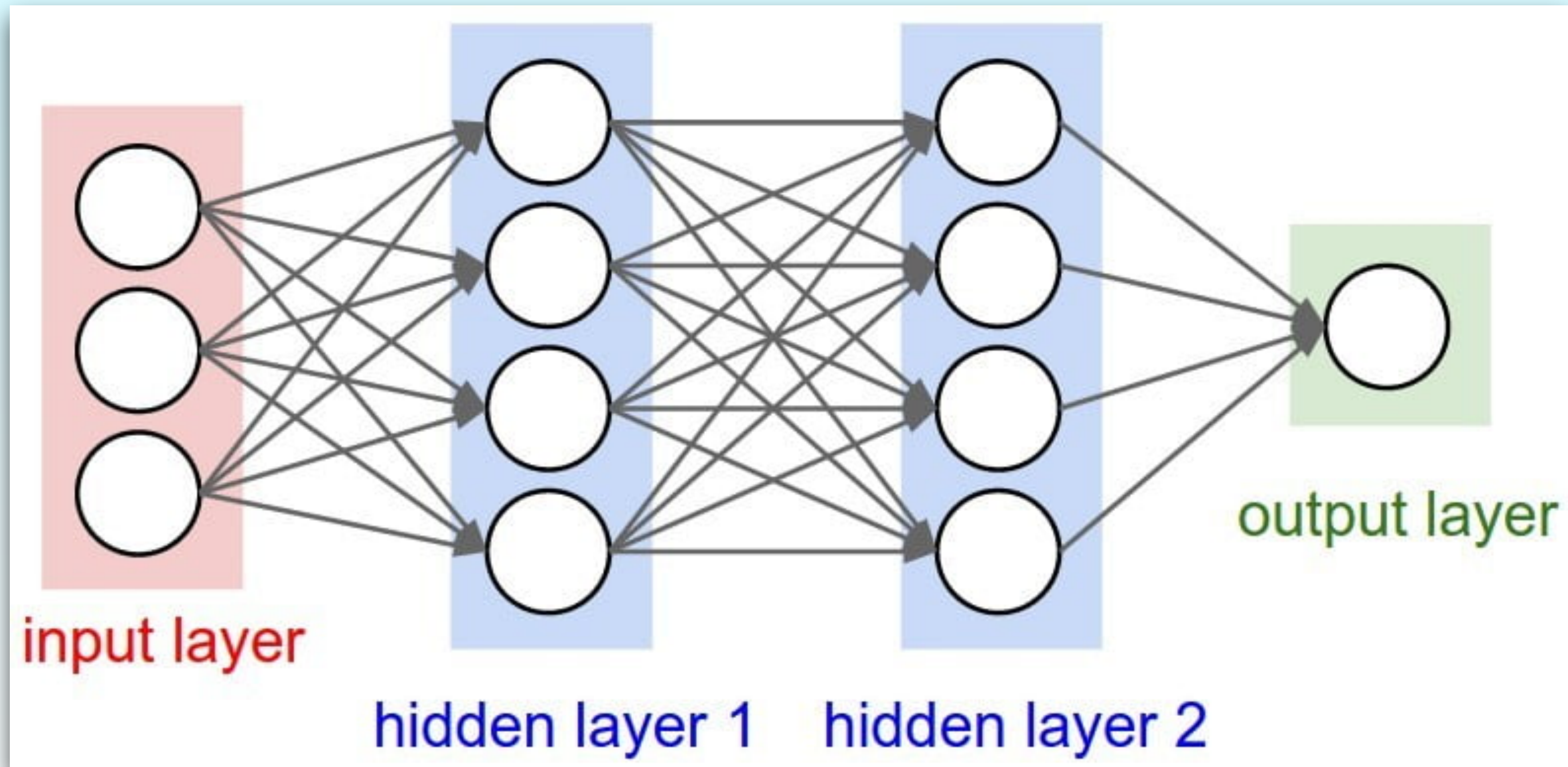
Transformers

LauzHack Deep Learning Winter Bootcamp

Seyed Parsa Neshaei — Feb 2025

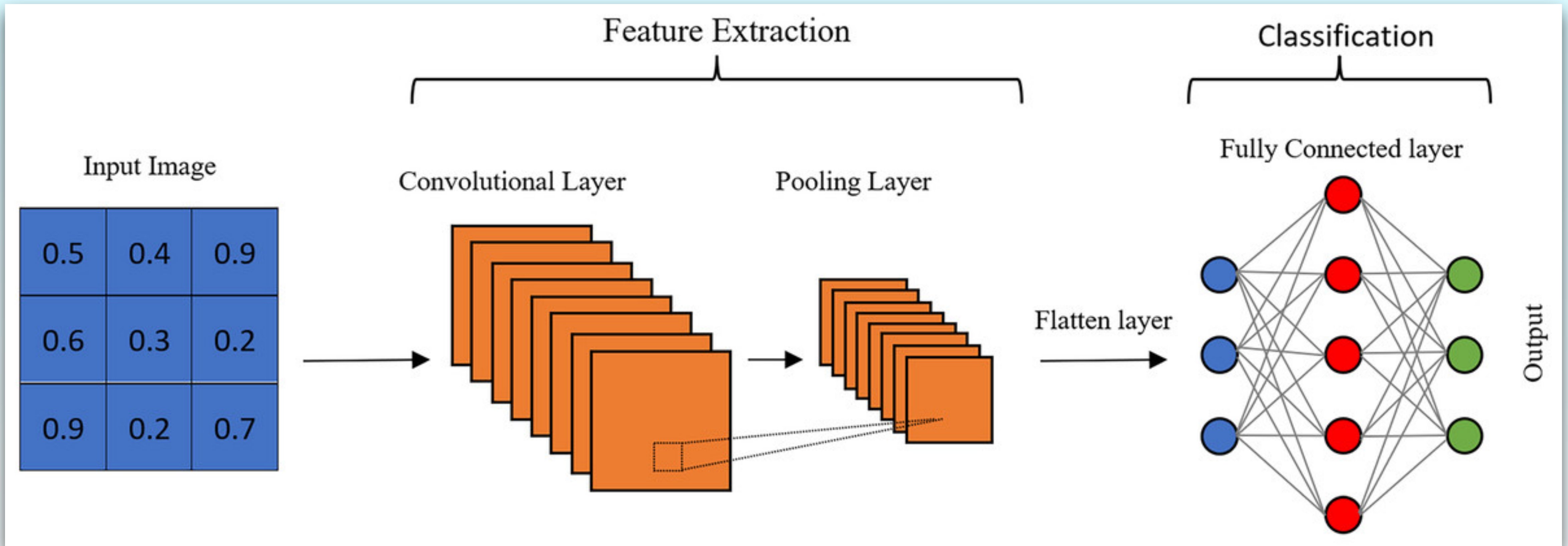
Let's have a recap!

Deep Neural Networks



Let's have a recap!

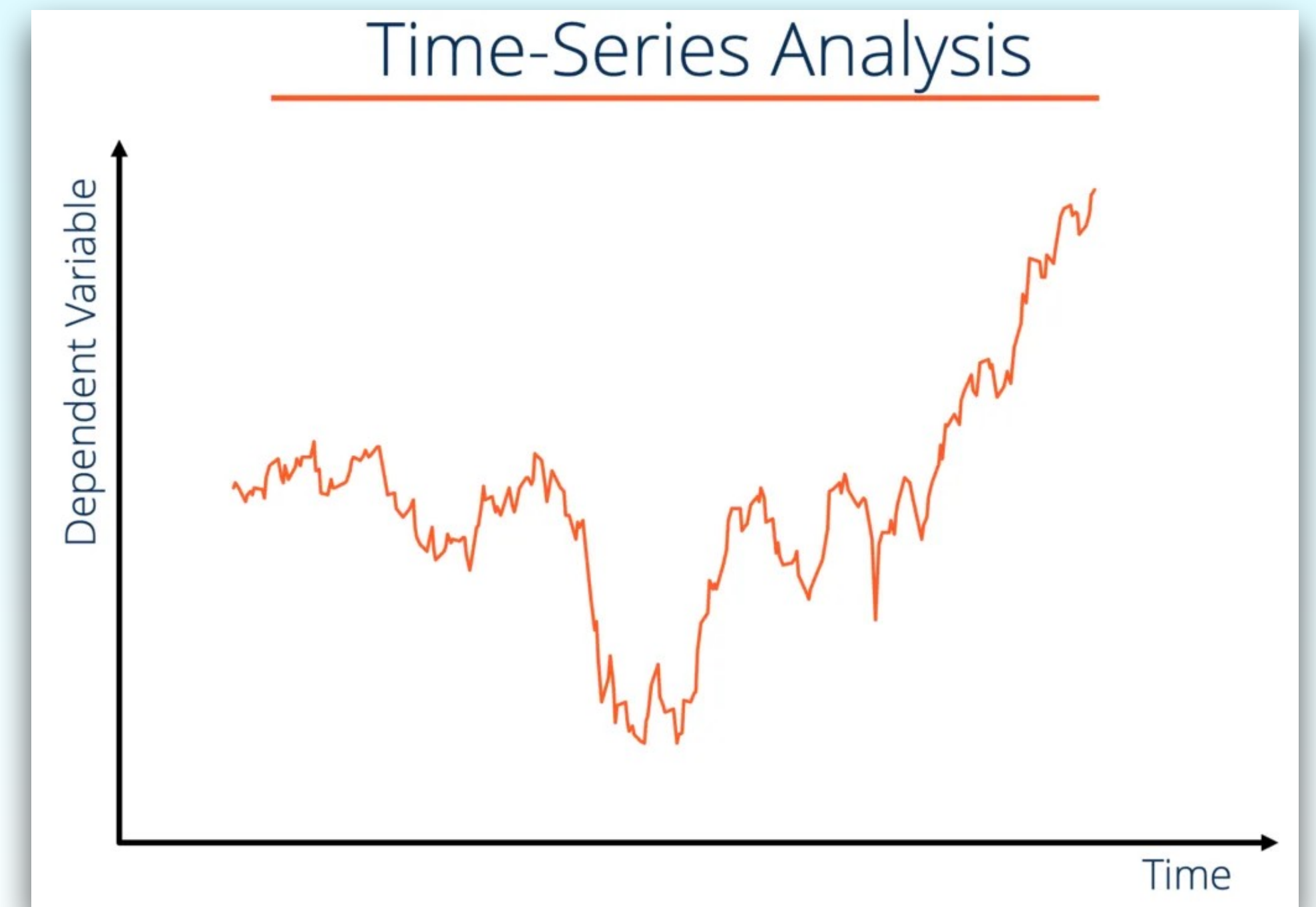
Convolutional Neural Networks



A new task!

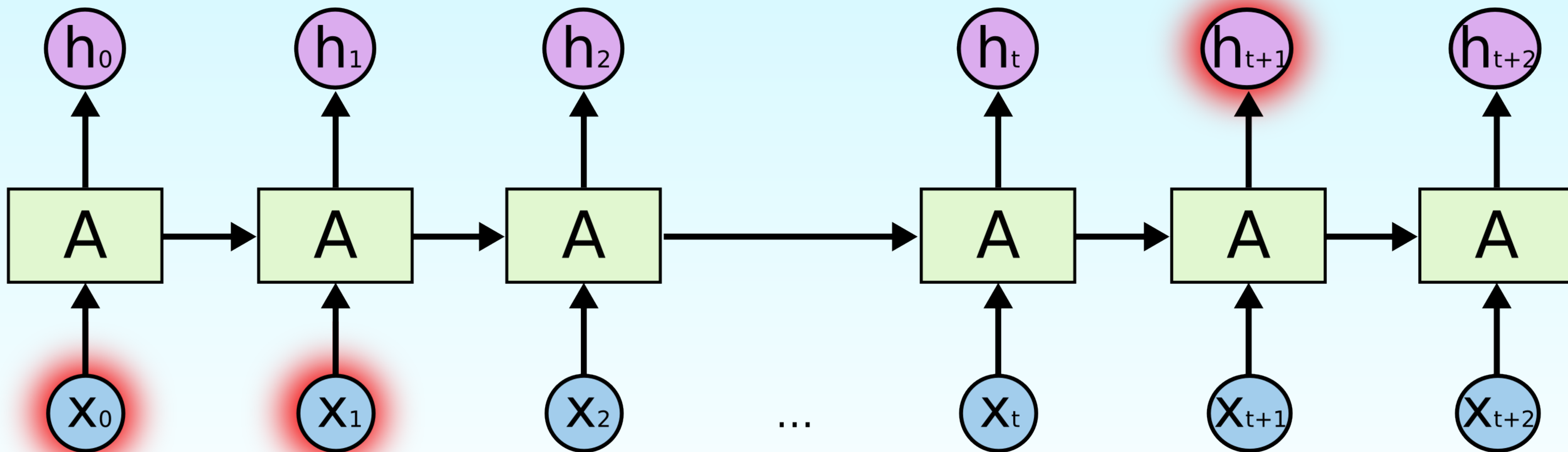
Time series

- Data measured at regular intervals, or step-by-step. Examples:
 - Hourly temperature in a city
 - Heights of ocean tides
 - Heartbeats per minute
 - The stock price of a company
 - Answers of students to exam questions
 - A written text (!): *EPFL I like* vs. *I like EPFL*
- Tasks: *classification* or *forecasting*
- Tip: if data semantics change by permutation



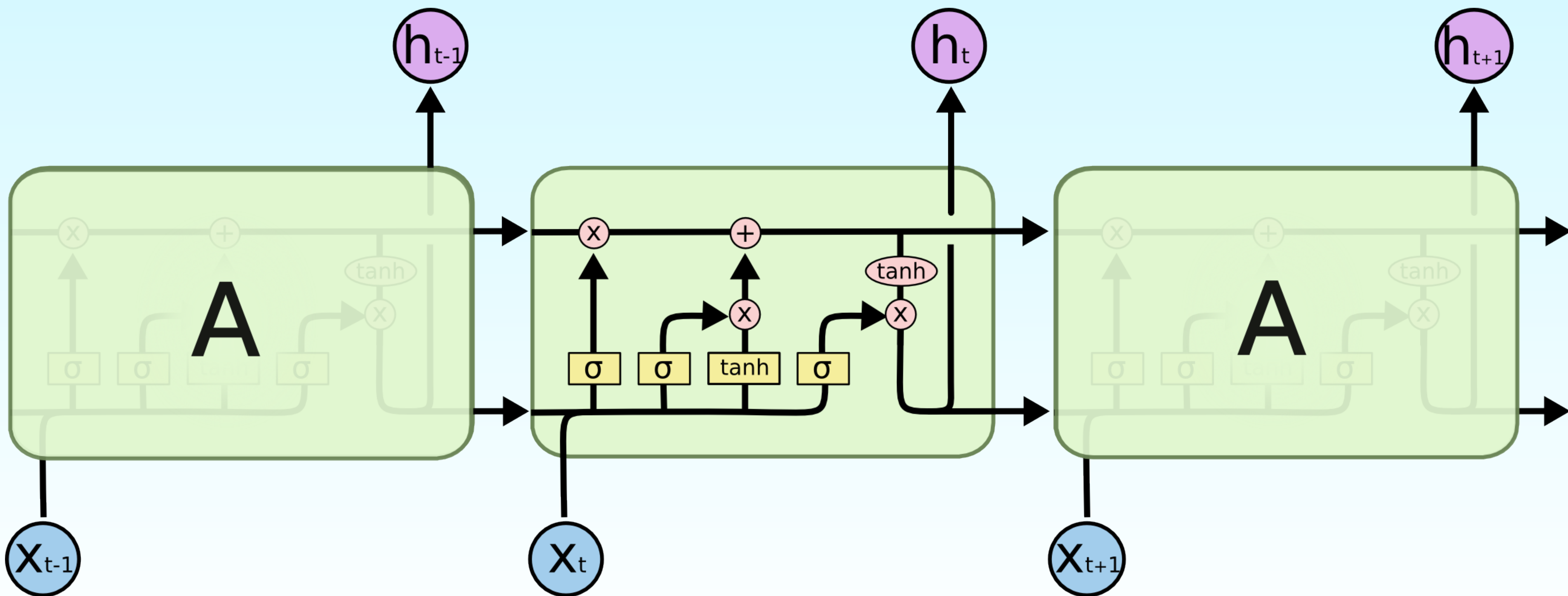
Let's have a recap!

Recurrent Neural Networks



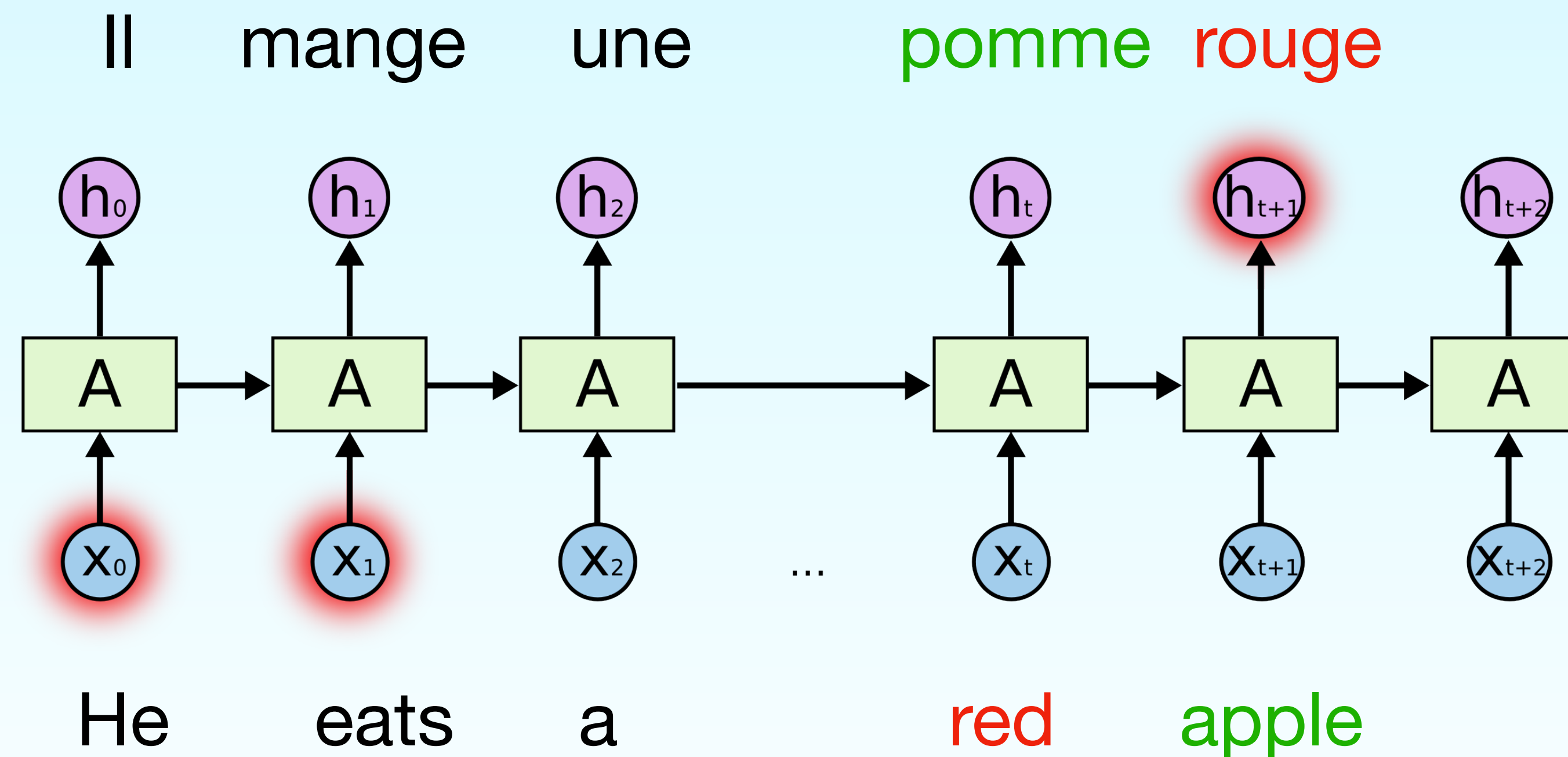
Let's have a recap!

Long-Short Term Memory Networks

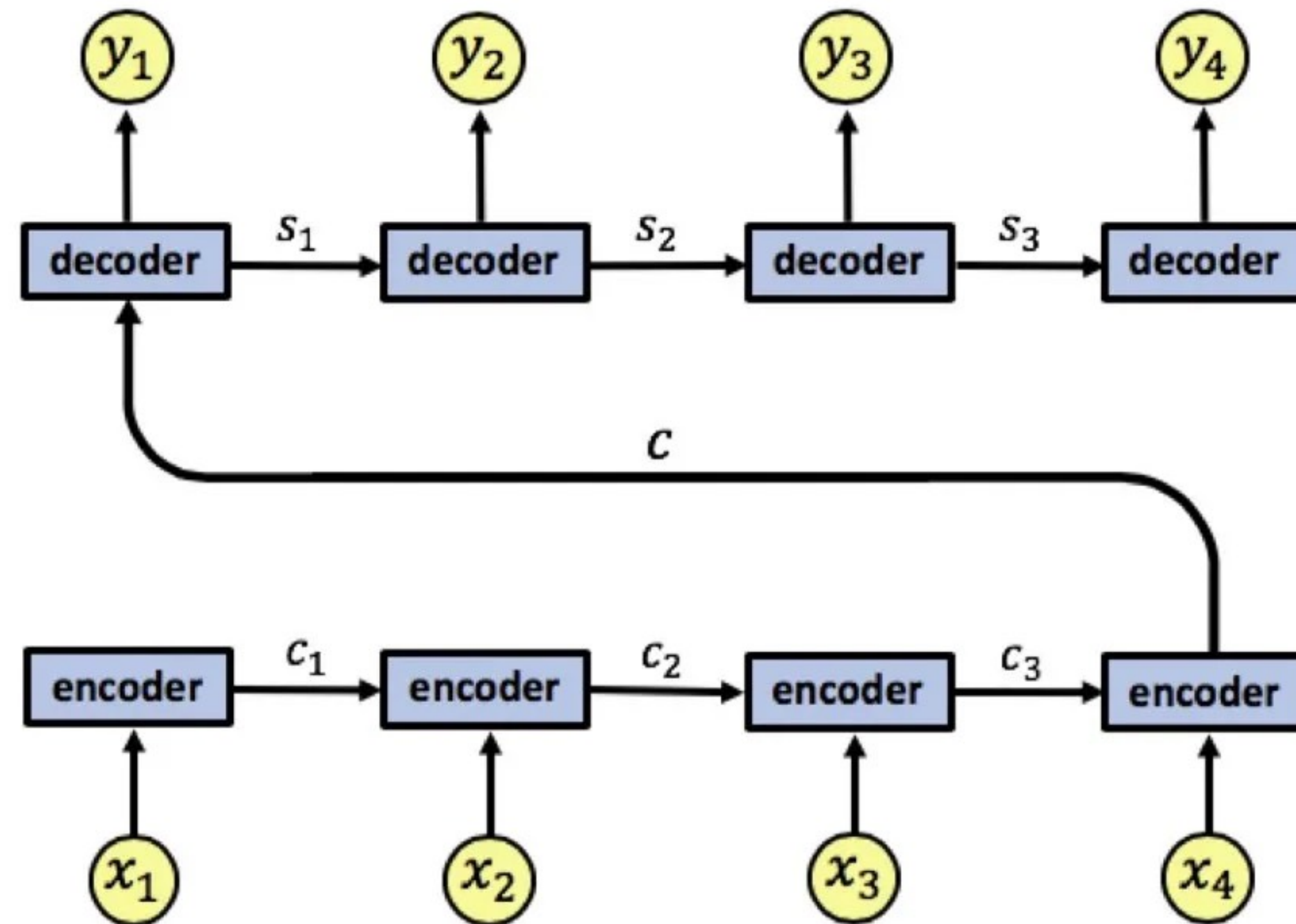


Let's have a recap!

Recurrent Neural Networks



Seq2seq Models



A new method today!

Question: which university/school is this?

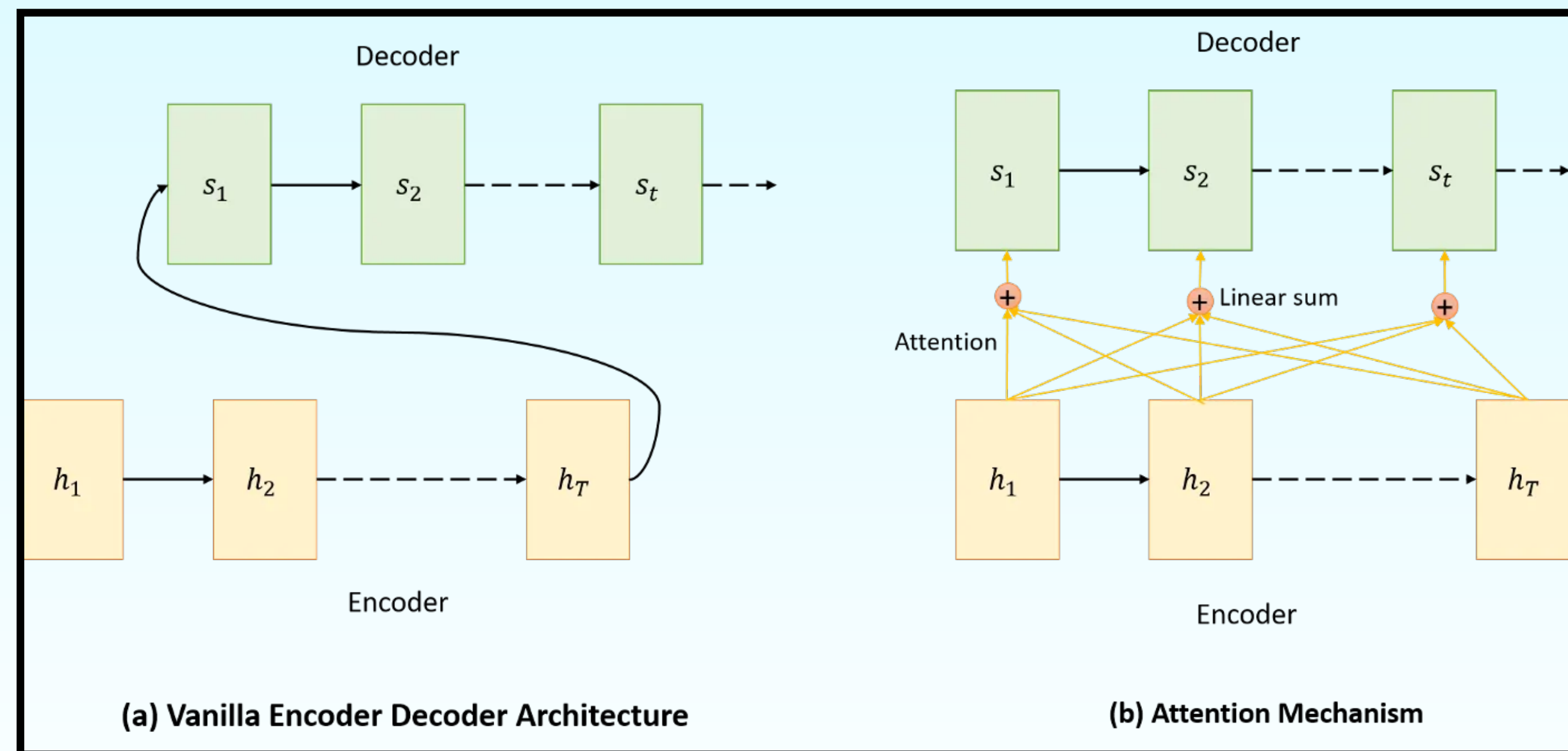
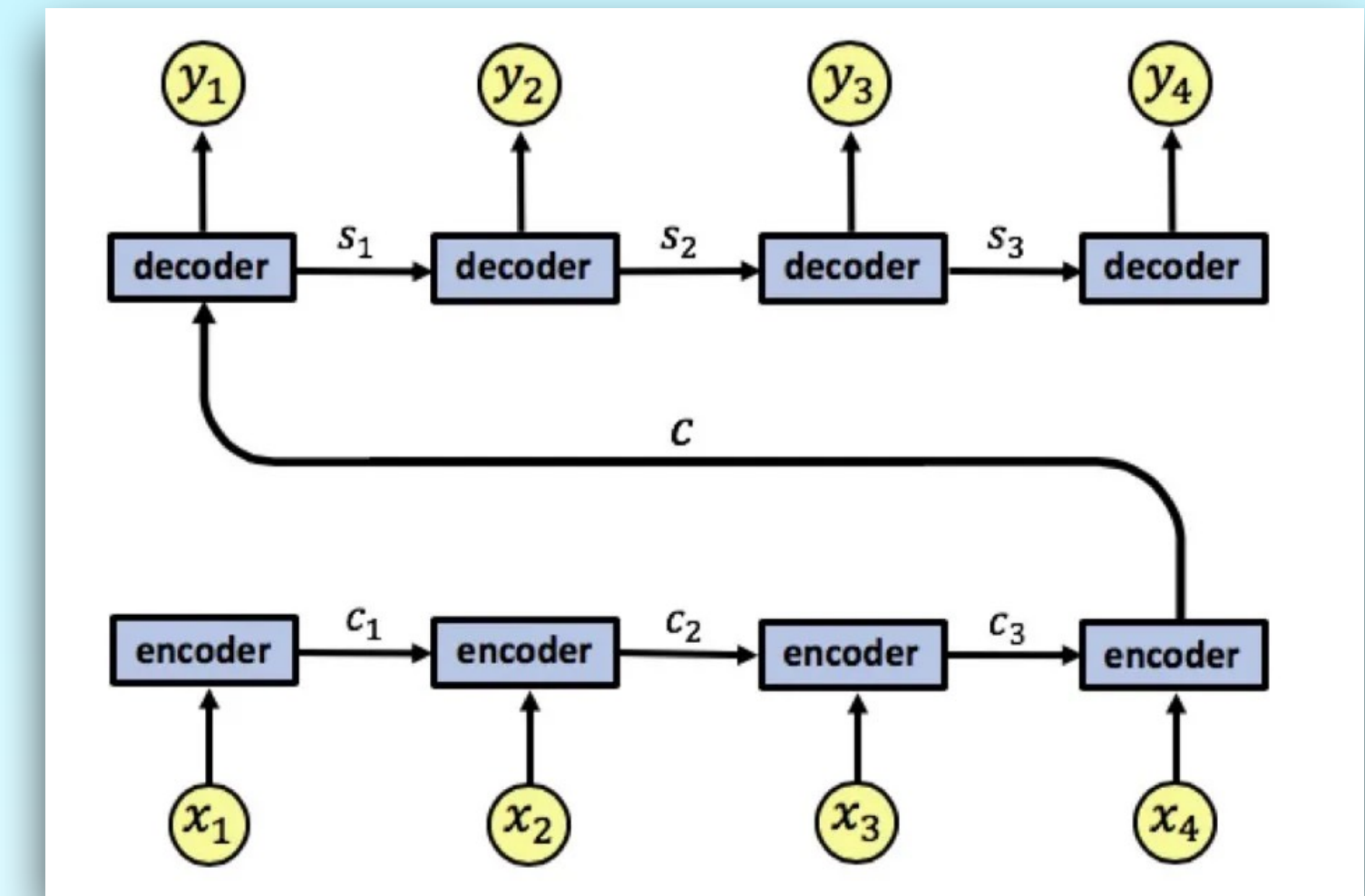


Question: classify the emotion (happy/sad)

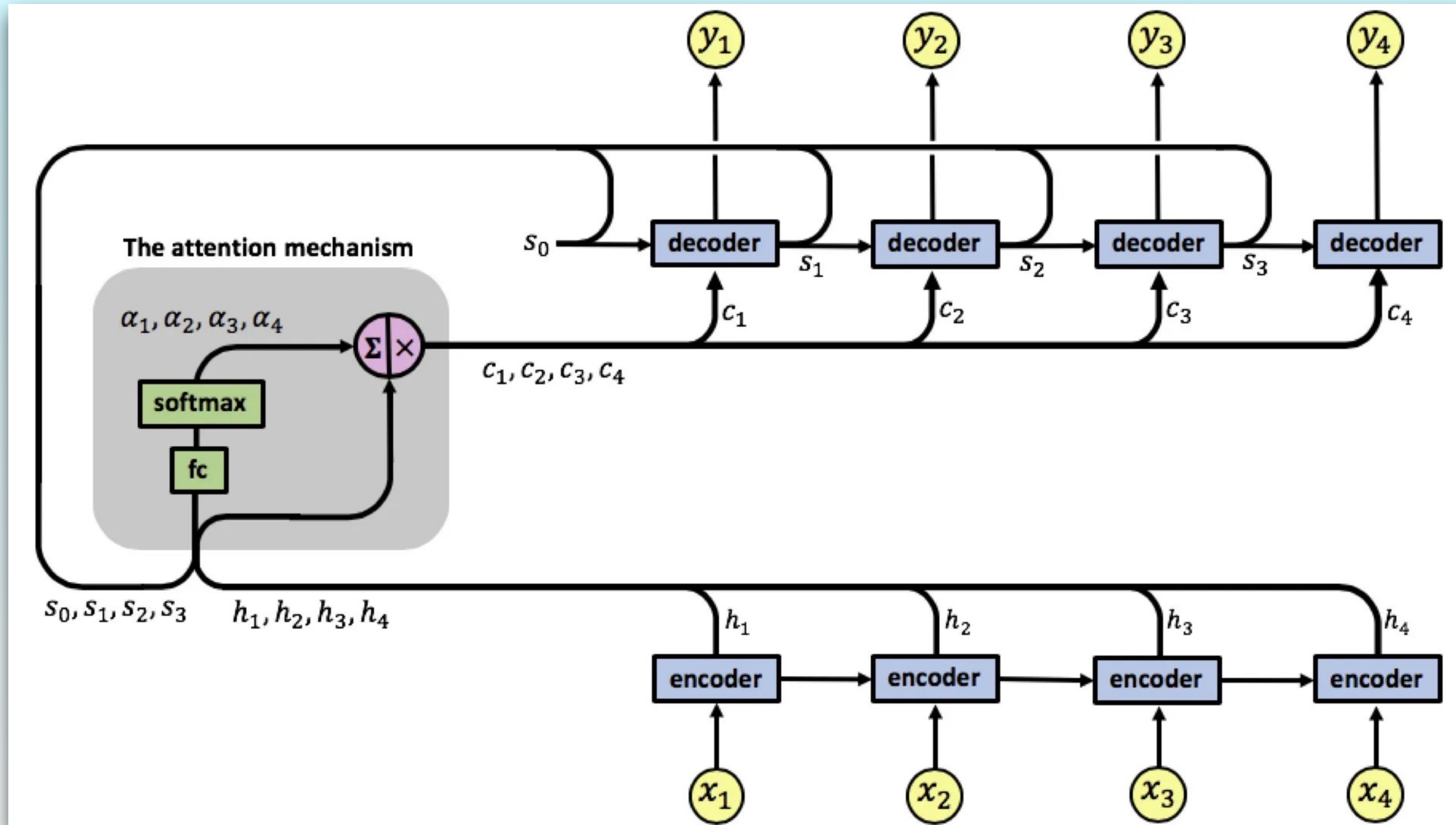
- Unfortunately, the professor would not have the meeting with us today.
- I'm thrilled to announce that I started a PhD at EPFL.

Attention Mechanism

- Until now, the final encoding steps had to represent the whole sequence into one vector
 - Information loss + the whole decoding from a single vector
- Idea: add an addition to the architecture, helping it to *attend* to the different inputs in the sequence
- Looks at all the encoder hidden states



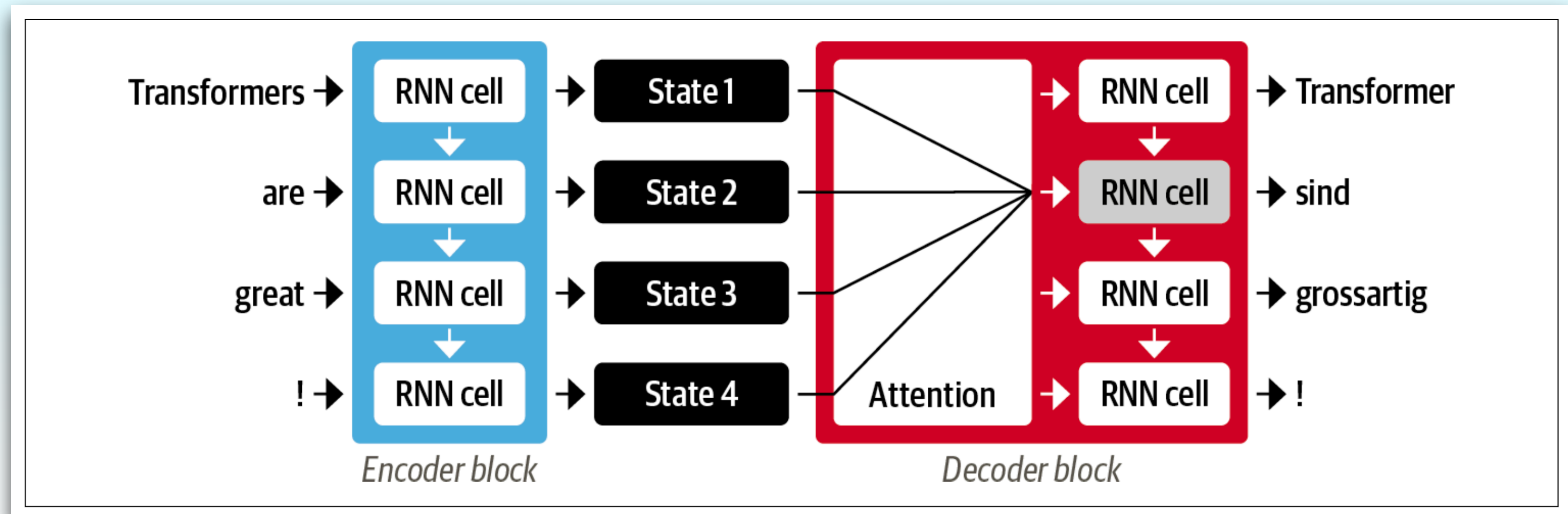
Attention Mechanism



Attention Mechanism

From Tunstall et al.:

“The main idea behind attention is that instead of producing a single hidden state for the input sequence, the encoder outputs a hidden state at each step that the decoder can access.”



Attention Mechanism

Learns non-trivial input alignments

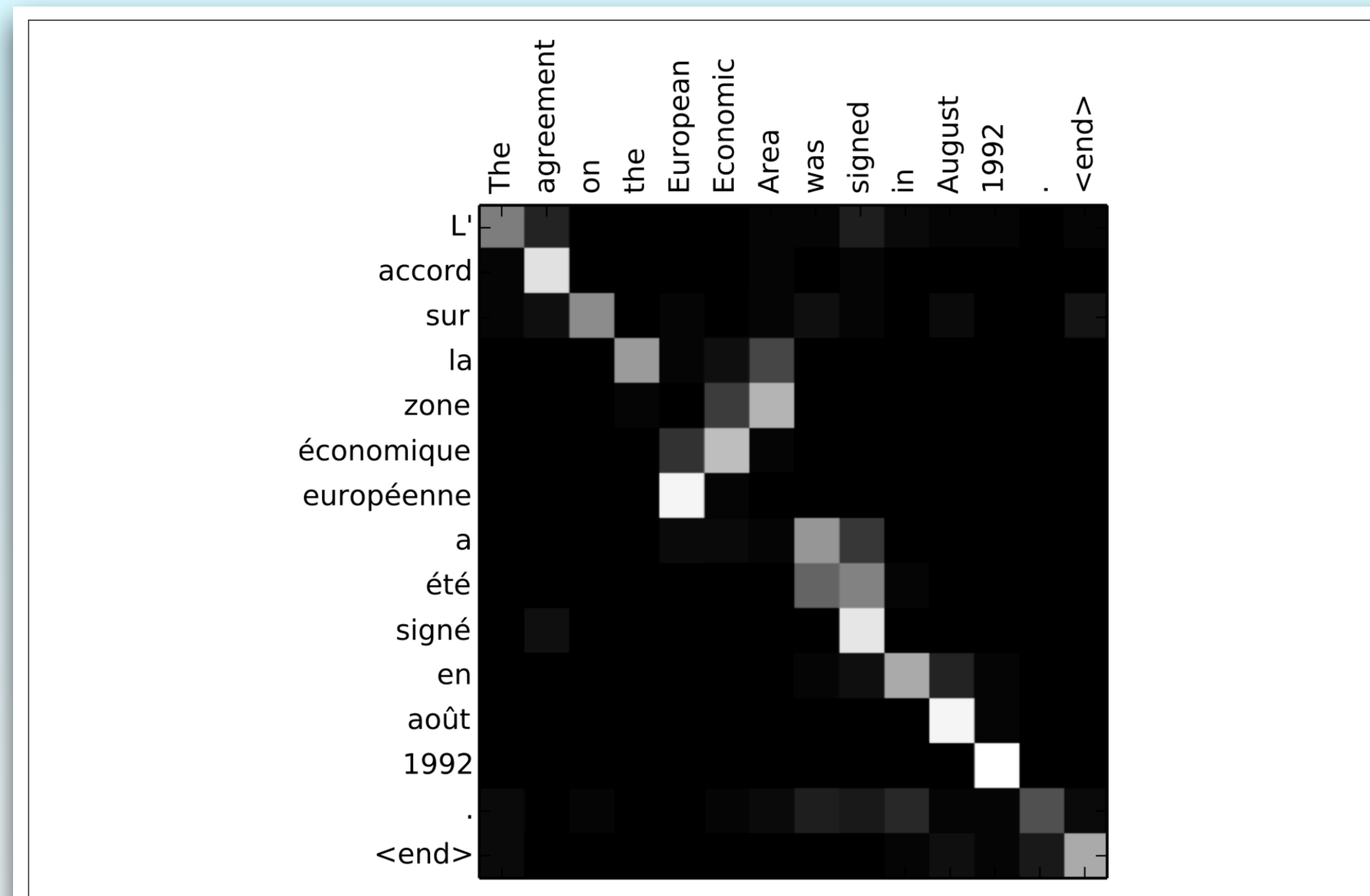
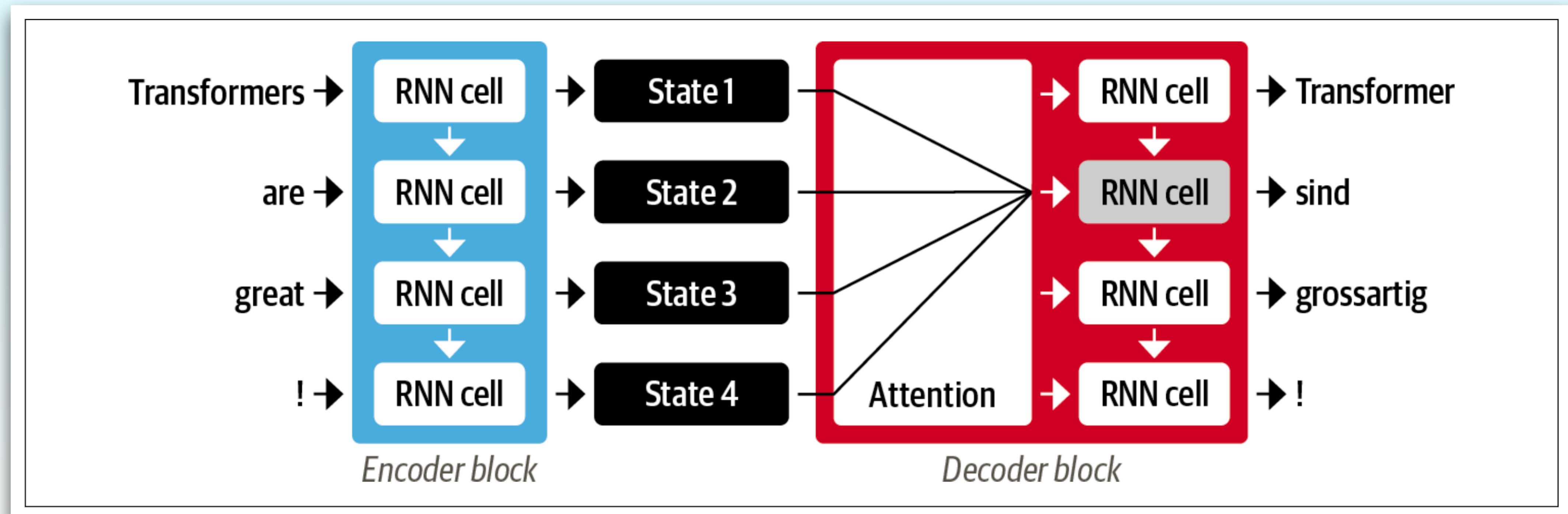


Figure 1-5. RNN encoder-decoder alignment of words in English and the generated translation in French (courtesy of Dzmitry Bahdanau)

Any downside that comes to mind?



Transformer

- Remove the recurrence! (Self-)attention is all you need!
- Encoder and Decoder (might be only one of them; see in NLP week!)
- Input can be text, image, ... (but has small details to take care of!)

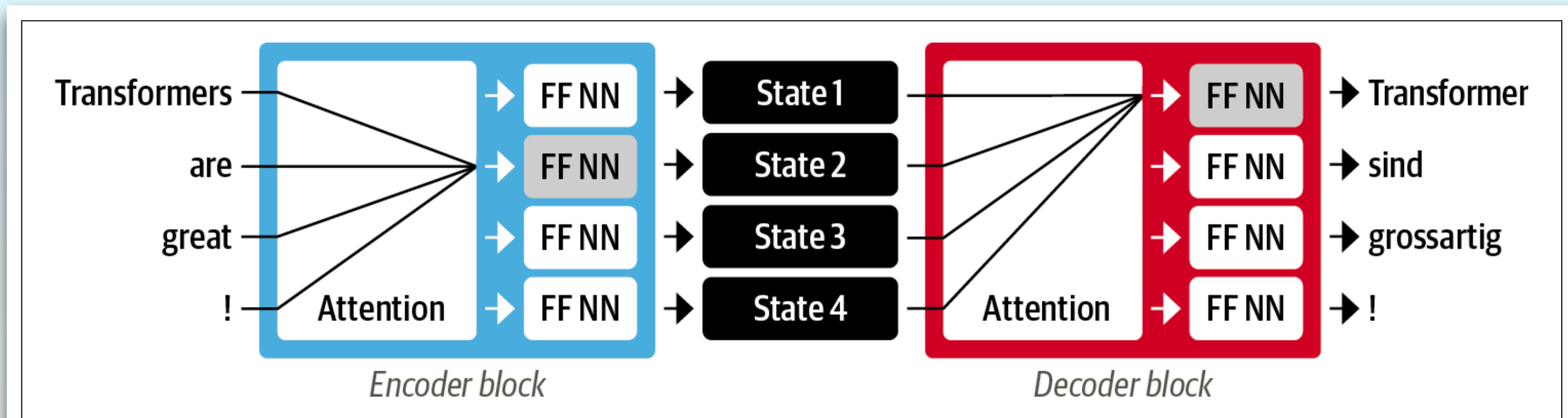
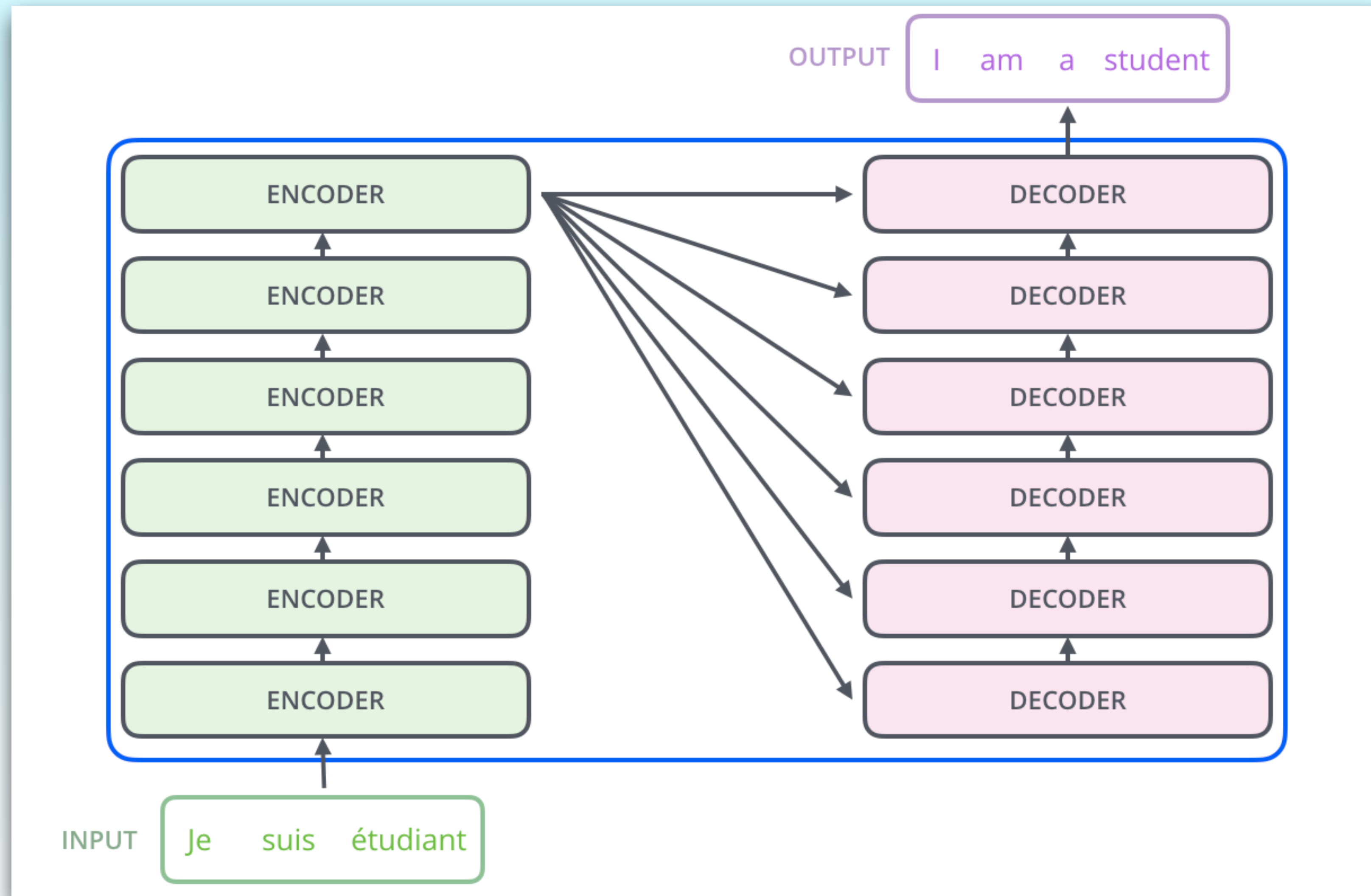


Figure 1-6. Encoder-decoder architecture of the original Transformer

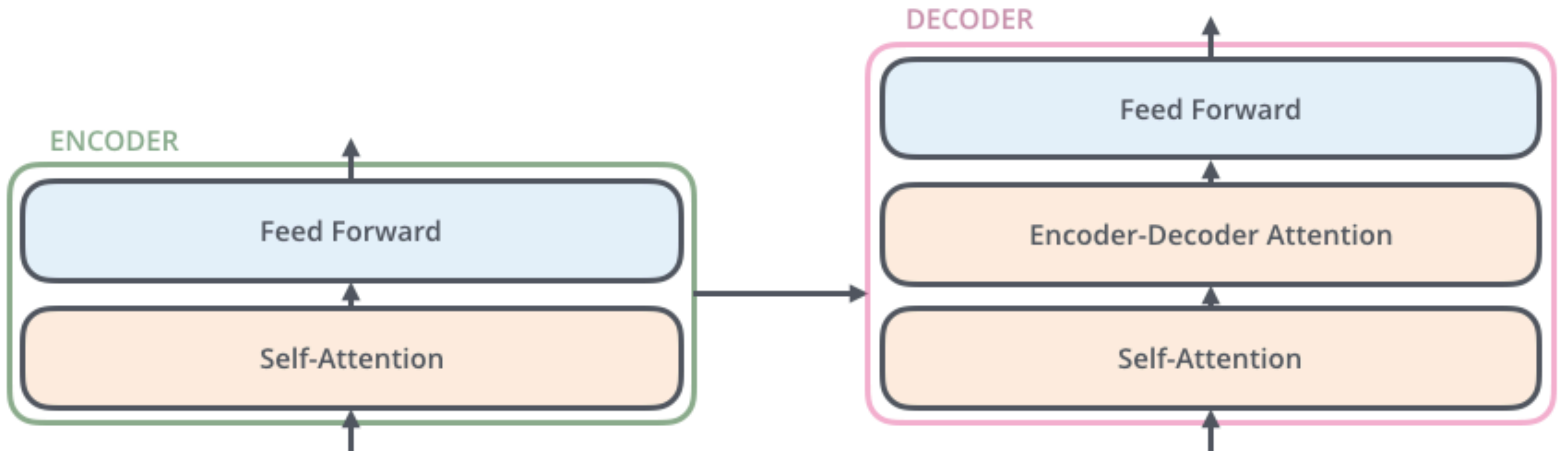
Let's explore how Transformers are designed!

Images and content in this section is from <https://jalammar.github.io/illustrated-transformer/>

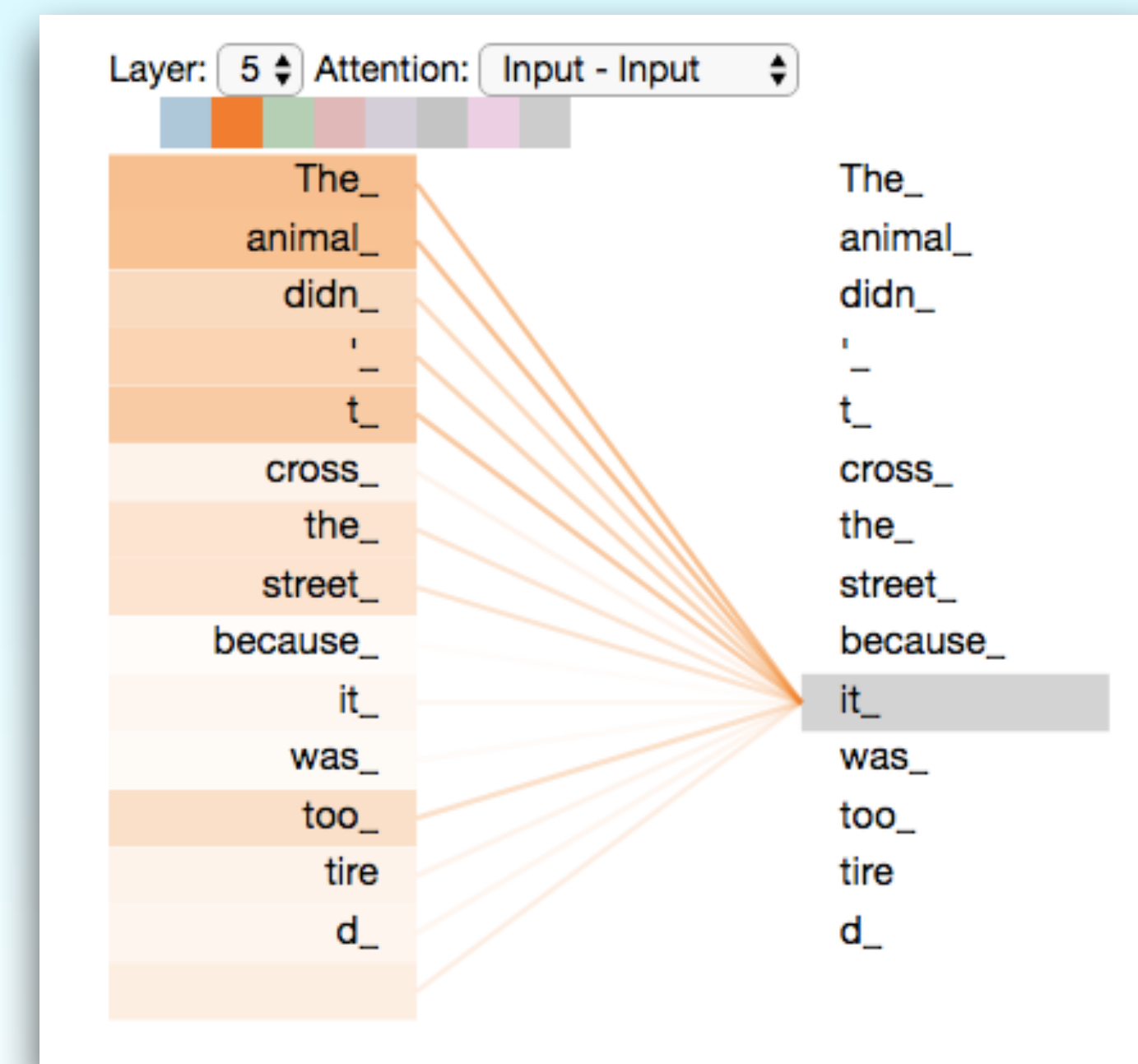
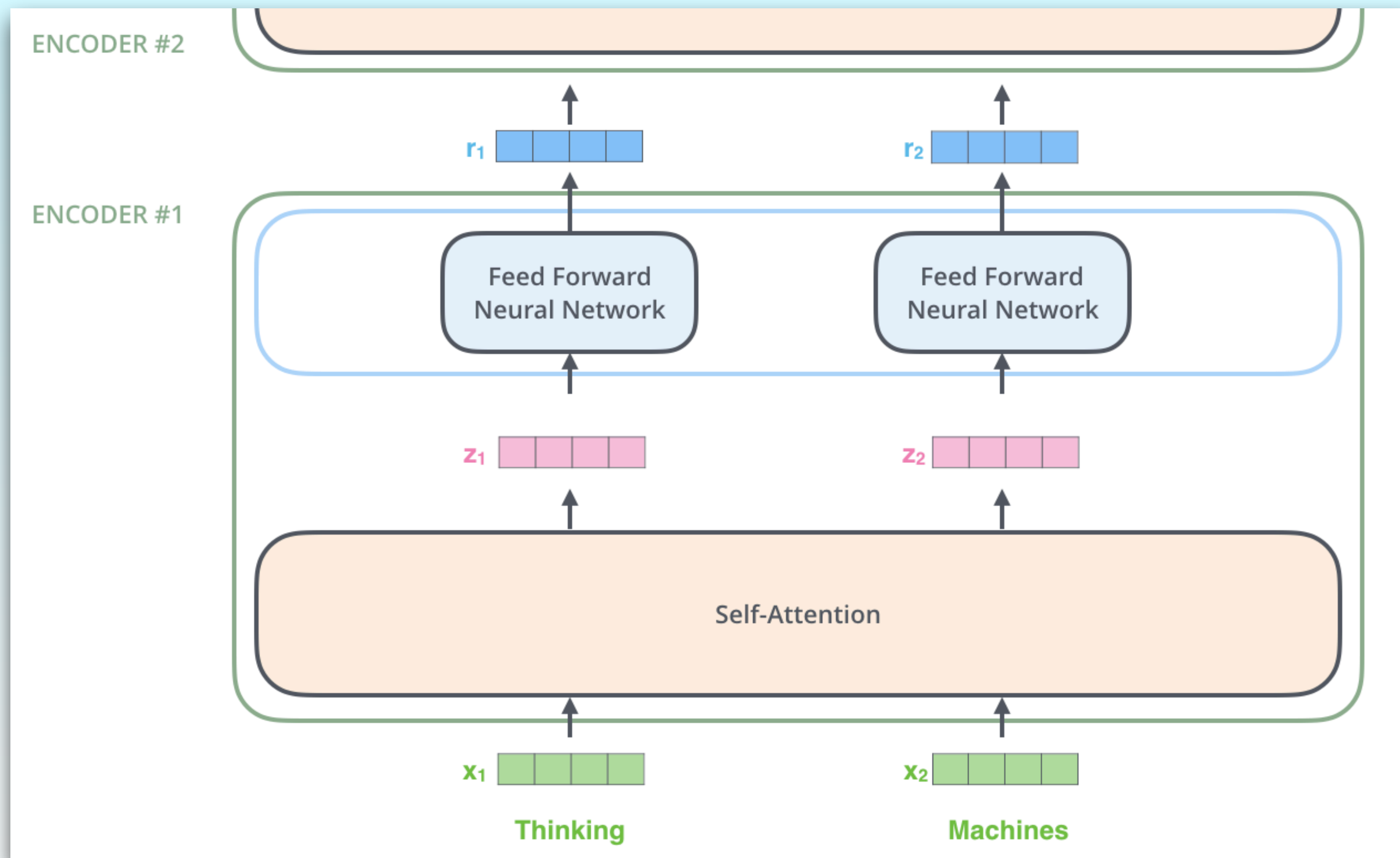
Transformer



Transformer



Transformer Encoder



Self-attention — find the differences!

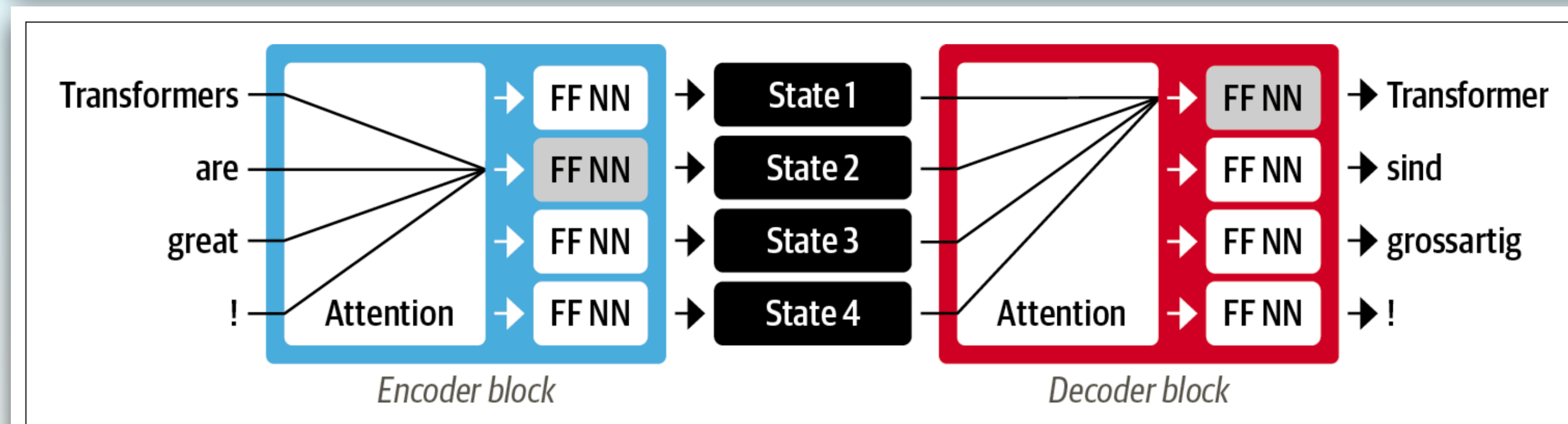
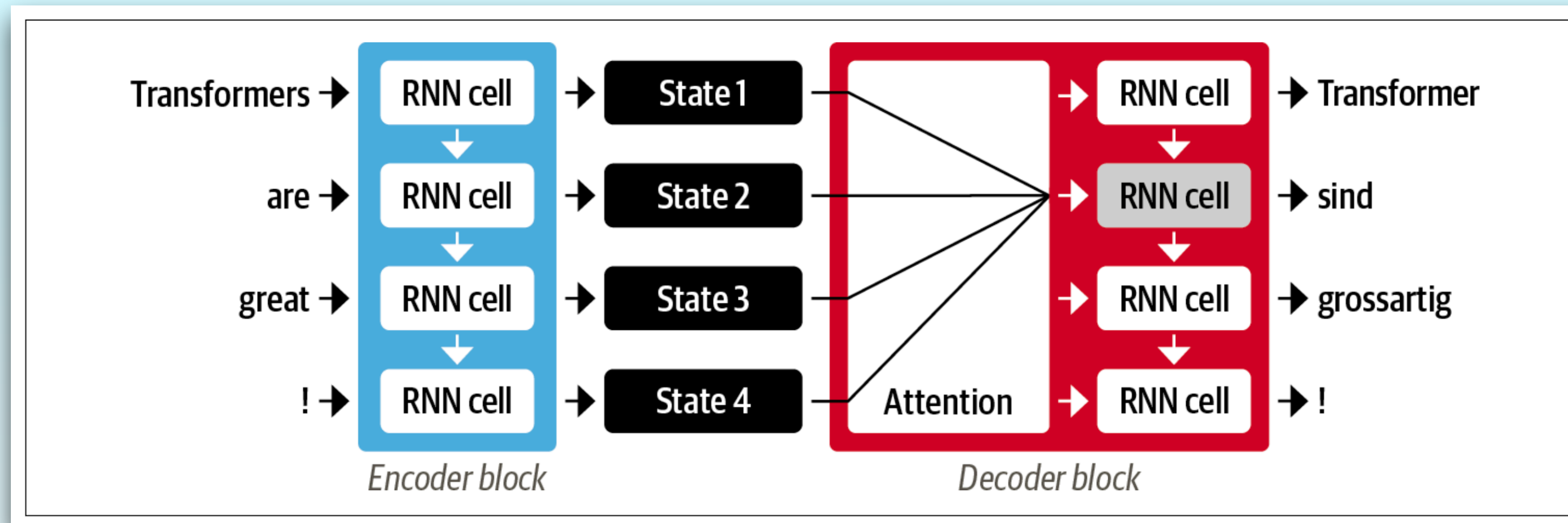
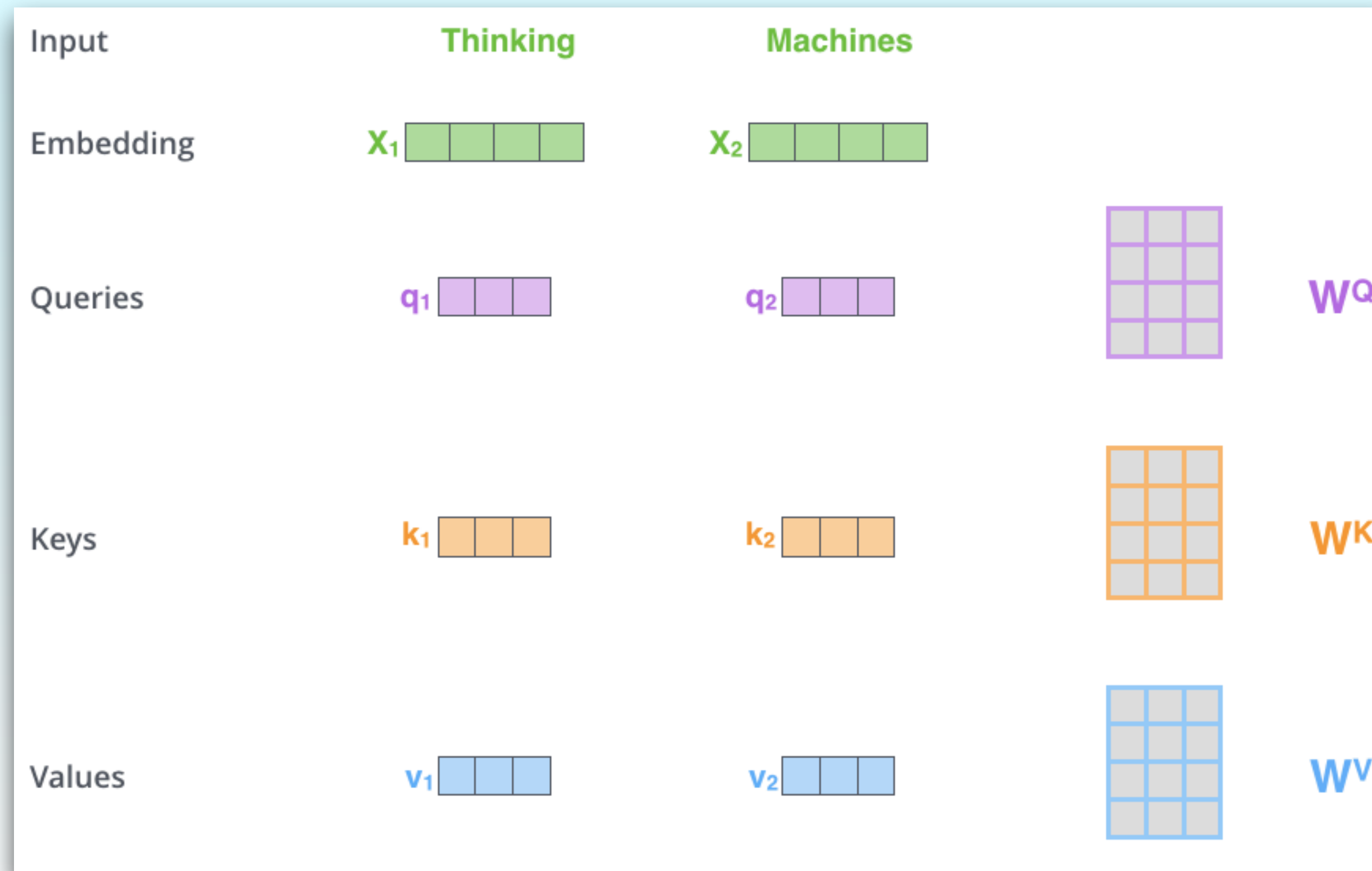


Figure 1-6. Encoder-decoder architecture of the original Transformer

Now let's get a bit more technical

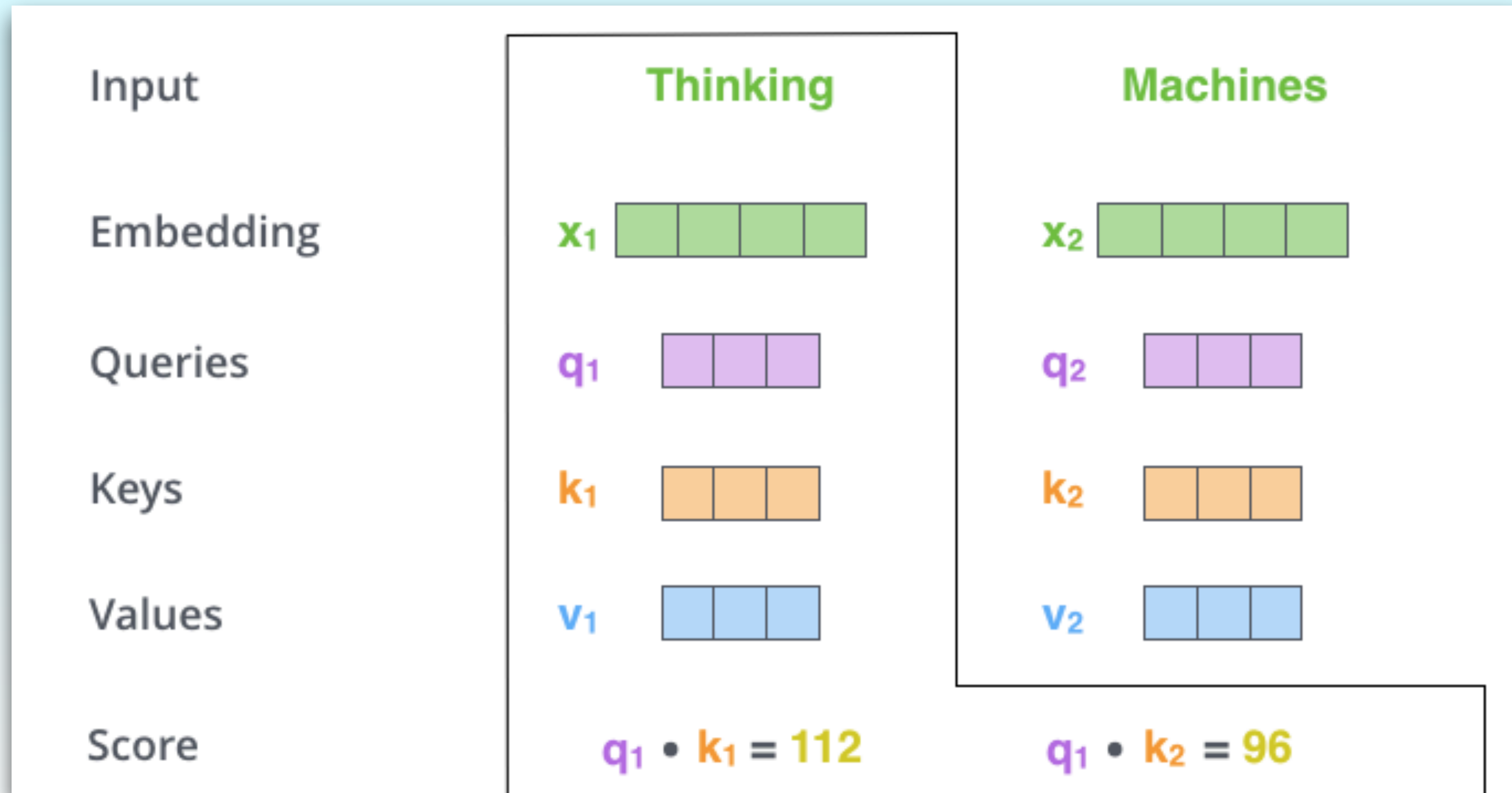
Self-attention Process

1) Create three vectors from each input of the encoder (can be smaller than input data!)



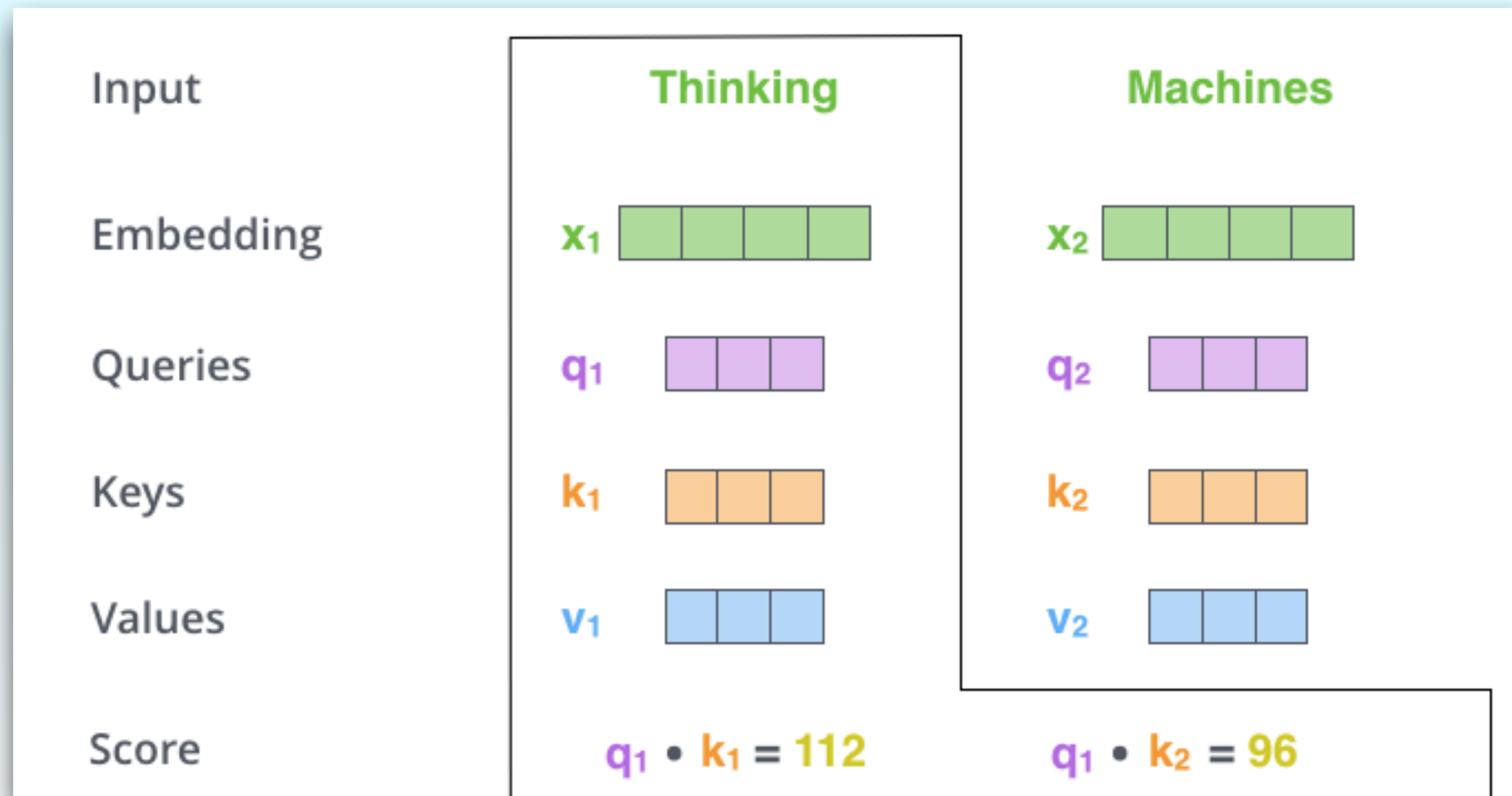
Self-attention Process

2) For each input, multiply its query with all the keys



Self-attention Process

3) Divide scores by sqrt of dim of key vectors (normalization) and put into softmax -> transforms the raw attention scores into a probability distribution over the input tokens

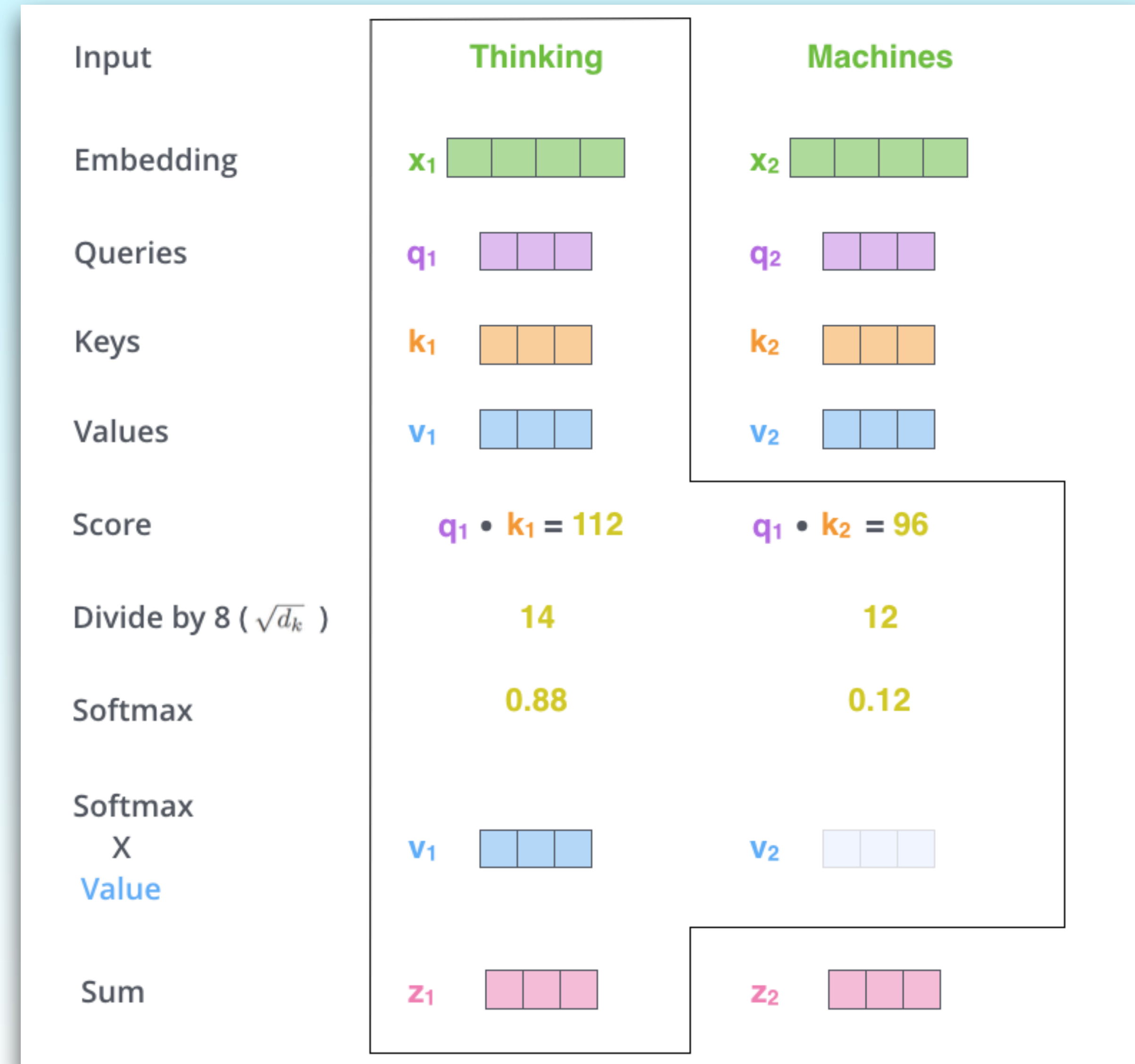


$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Self-attention Process

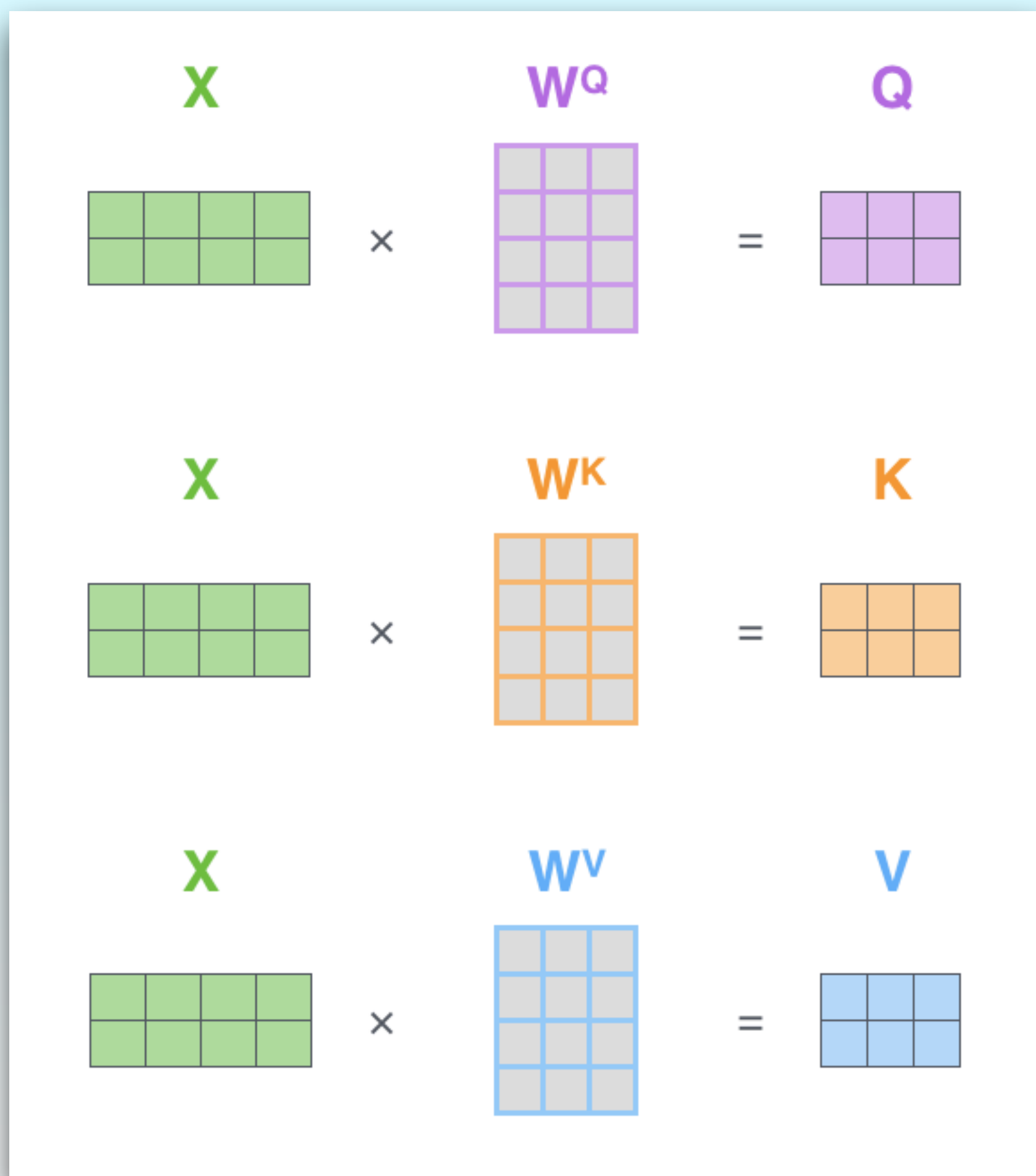
4) Multiply *value* vectors by softmax,
then sum up

Done!



Self-attention In Practice: Matrices

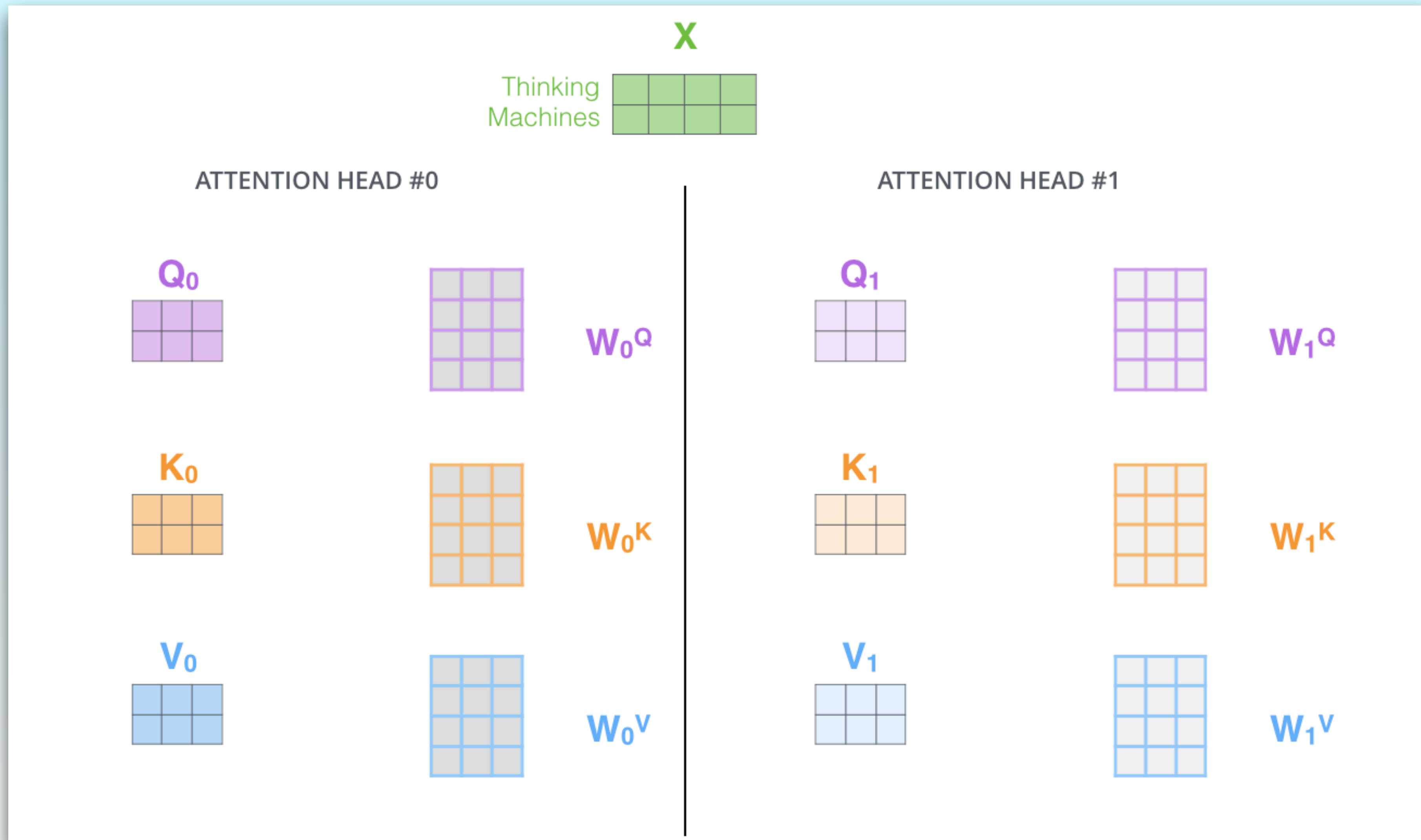
All embeddings into X ...!



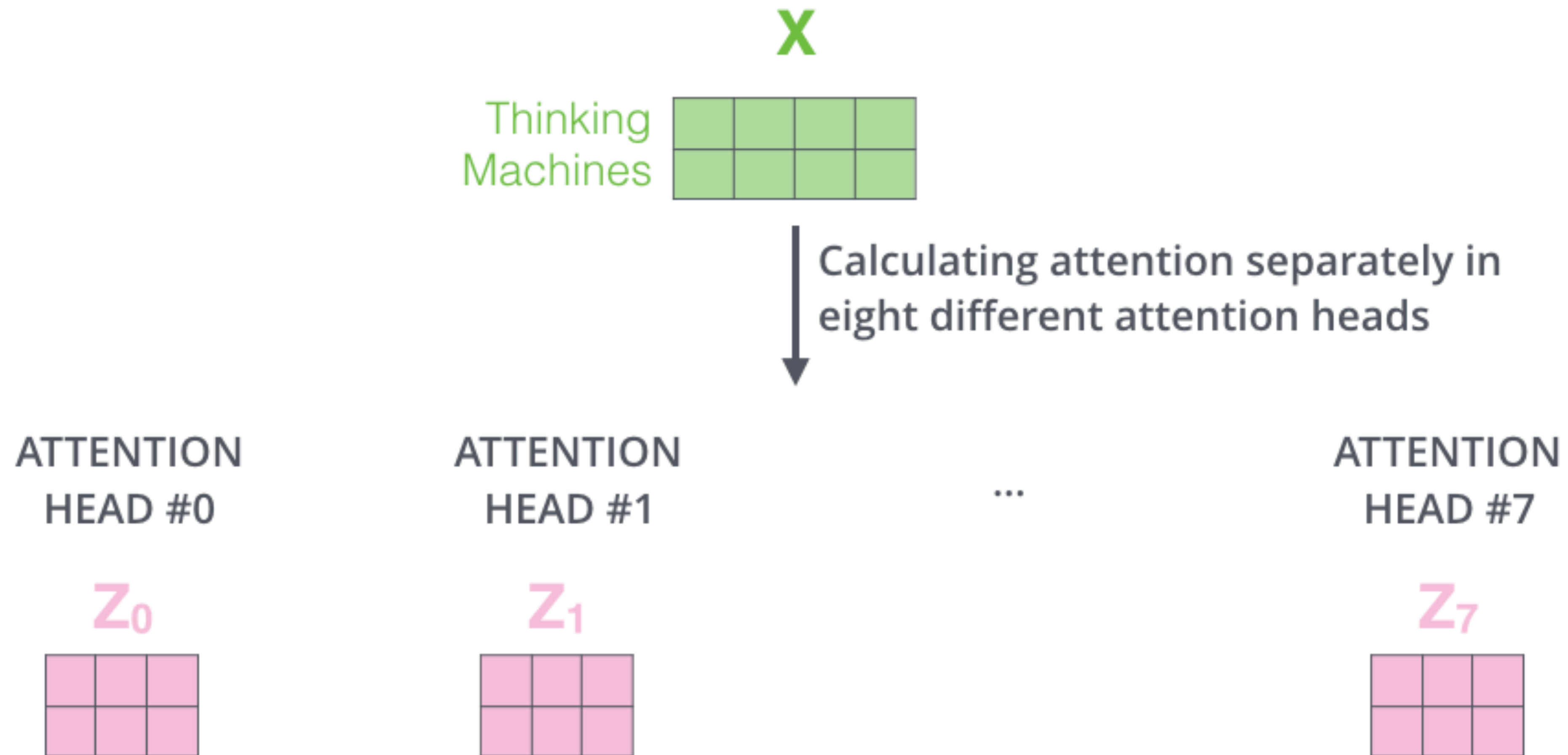
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

The diagram shows the calculation of the attention matrix Z (pink, 2x3). It involves the softmax function applied to the scaled dot product of Q (purple, 2x3) and K^T (orange, 3x2). The result is then multiplied by V (blue, 2x3) to produce Z .

Can be multi-headed

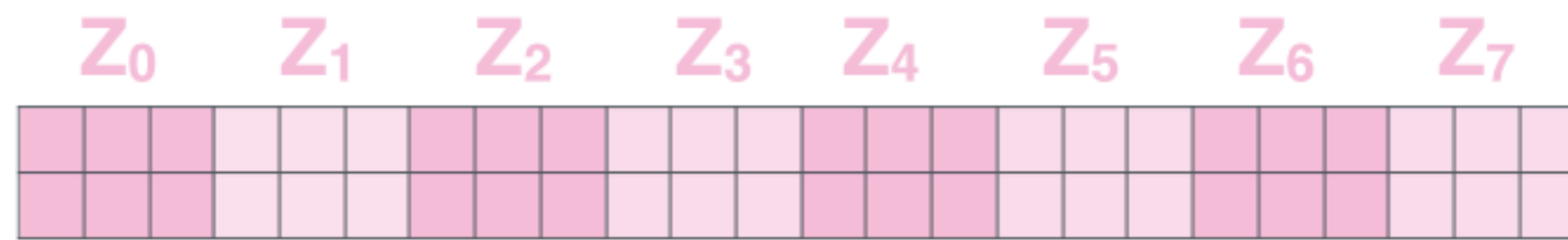


Can be multi-headed



Can be multi-headed

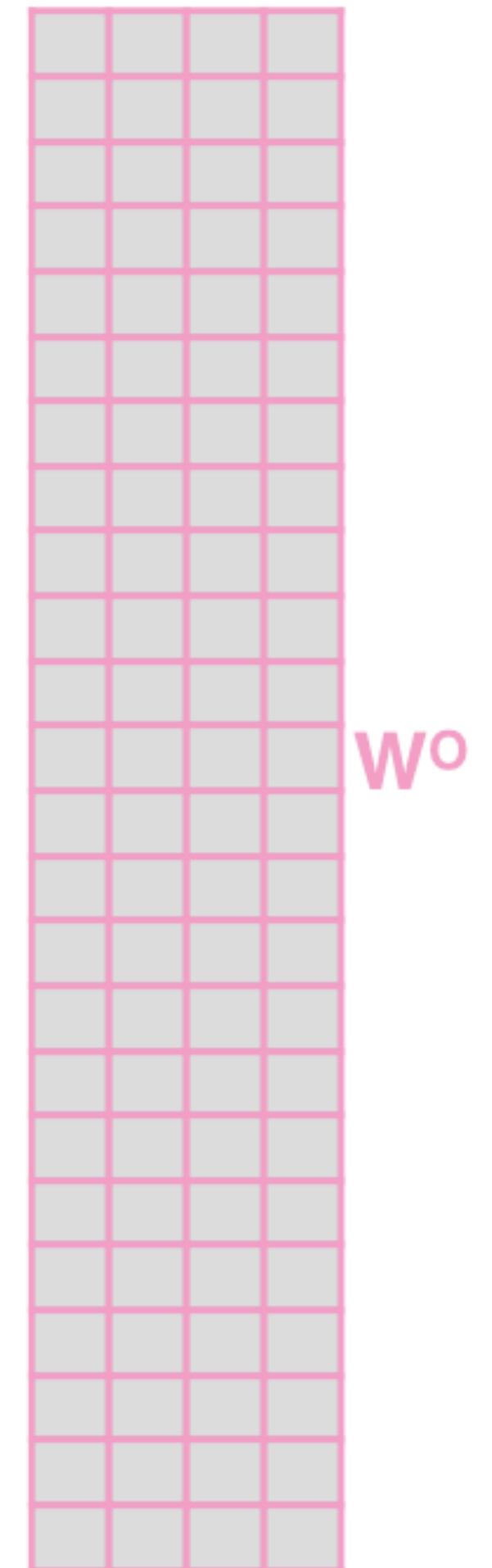
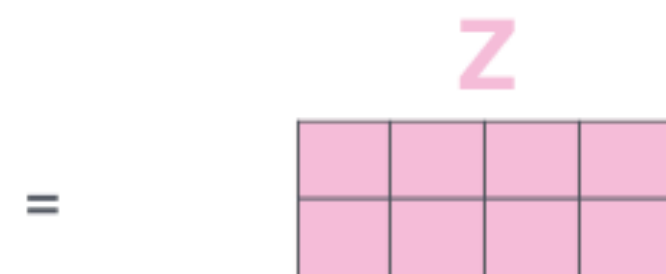
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

X

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



What is one problem with this approach?

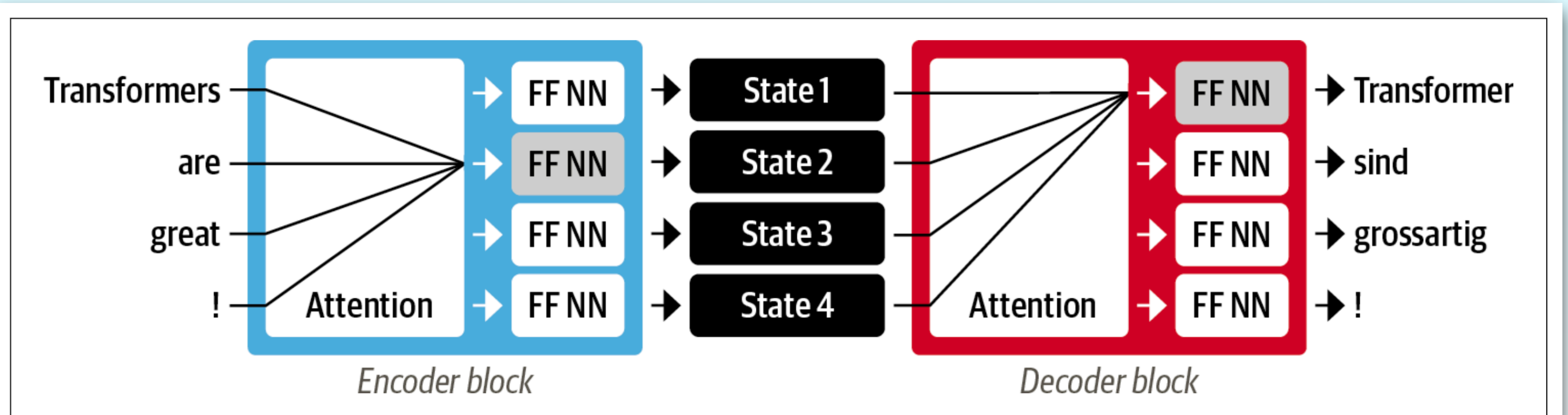
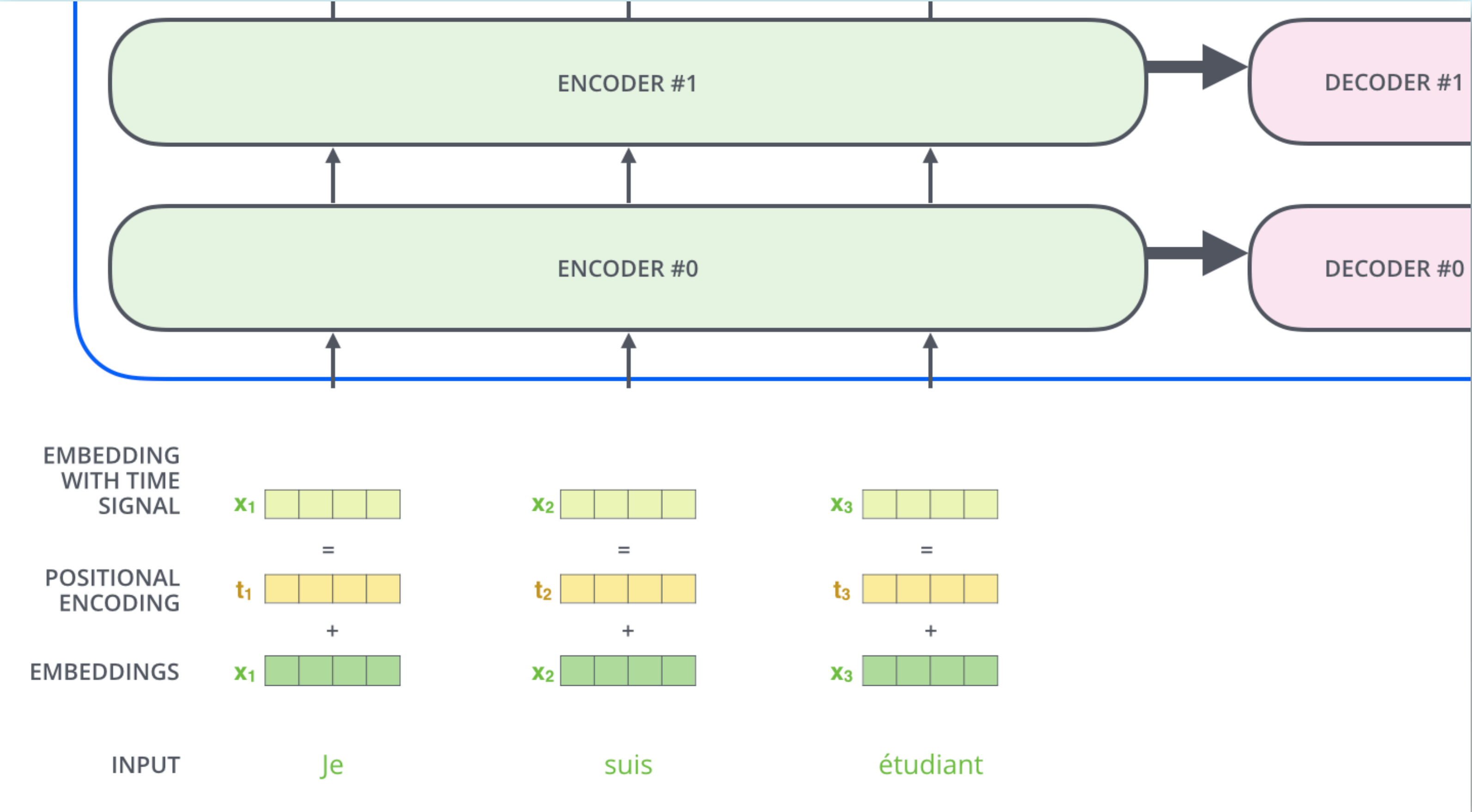


Figure 1-6. Encoder-decoder architecture of the original Transformer

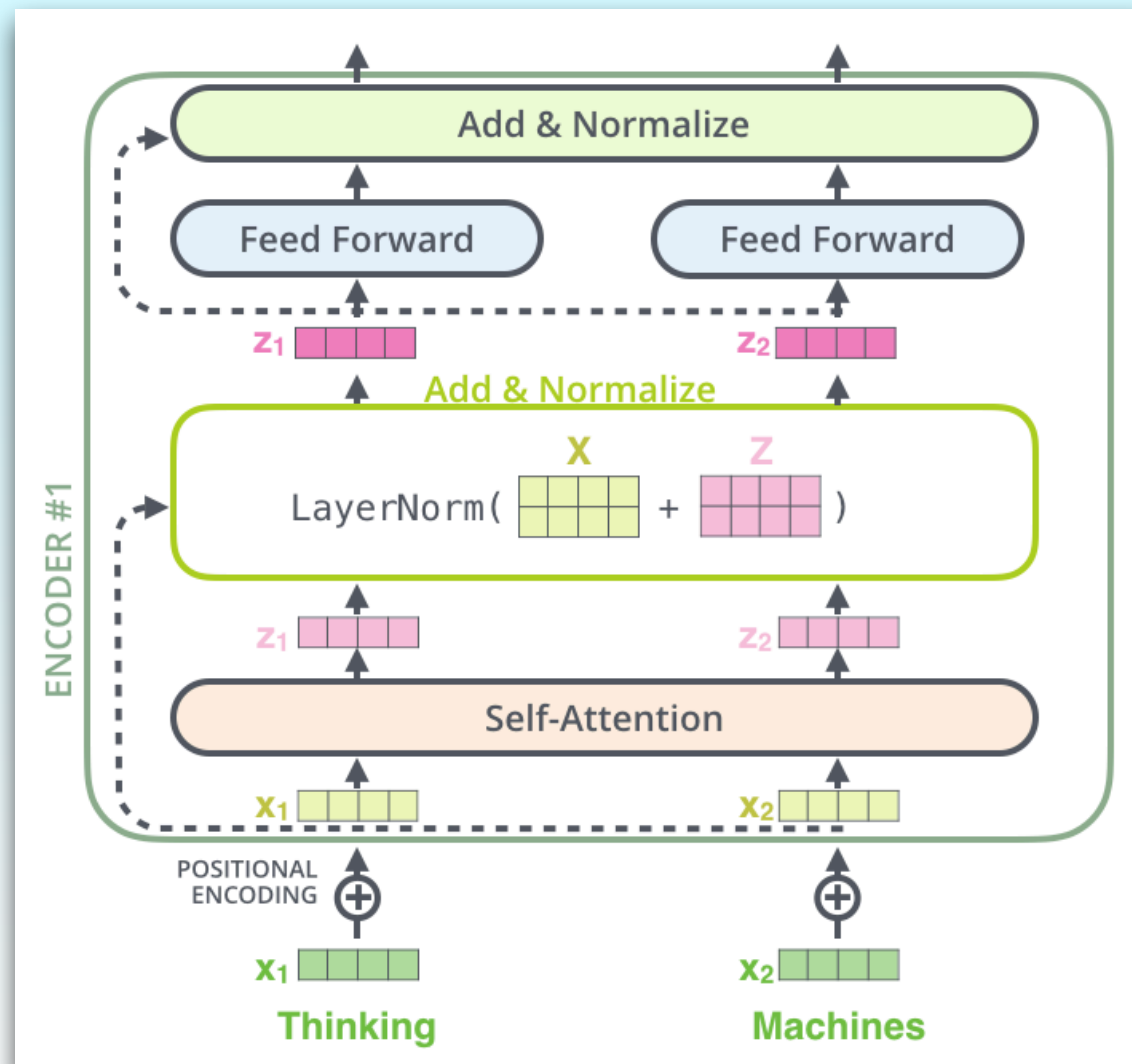
Positional Encoding

✓ Pros and Cons

	Trainable Positional Encoding	Sinusoidal Positional Encoding
Learned	Yes	No
Generalizes to longer sequences?	No, limited to training length	Yes, since it's a function
Flexible?	Yes, learns task-specific patterns	Less flexible



Full Encoder Together!



Batch Normalization

What is it?

- A technique to improve model generalizability and speeds up training (enables higher learning rates)
- Find the mean and SD of mini-batches of inputs in training, then normalize
- Also makes the model more robust to the initial initialization
- Just add after activation functions
- People do NOT suggest its use for RNNs!

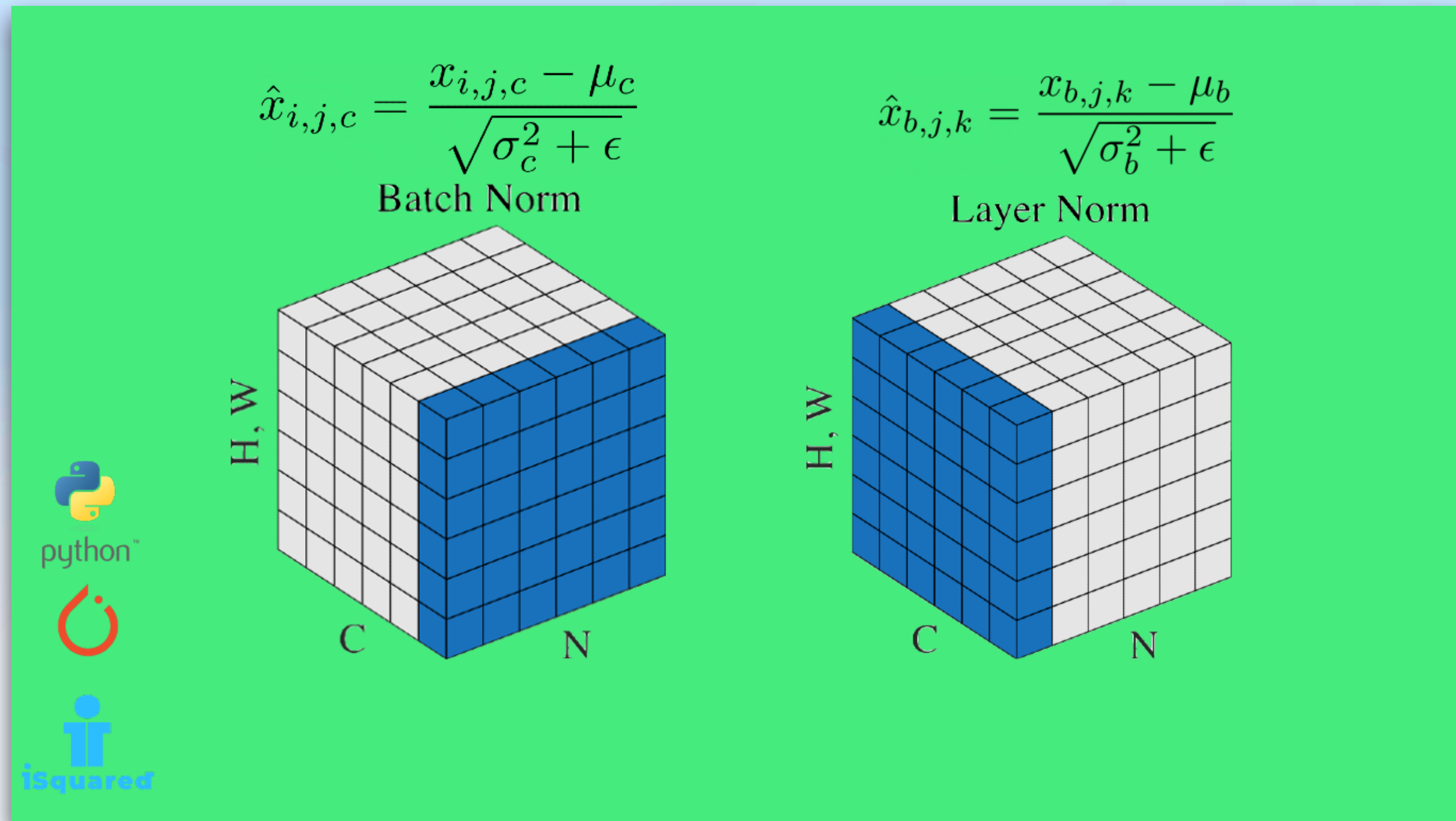
```
class MyModel(nn.Module):  
    def __init__(self):  
        super(MyModel, self).__init__()  
        self.fc1 = nn.Linear(128, 64)  
        self.bn1 = nn.BatchNorm1d(64)  
        self.relu = nn.ReLU()  
        self.fc2 = nn.Linear(64, 10)
```

```
def forward(self, x):  
    x = self.fc1(x)  
    x = self.bn1(x)  
    x = self.relu(x)  
    x = self.fc2(x)  
    return x
```

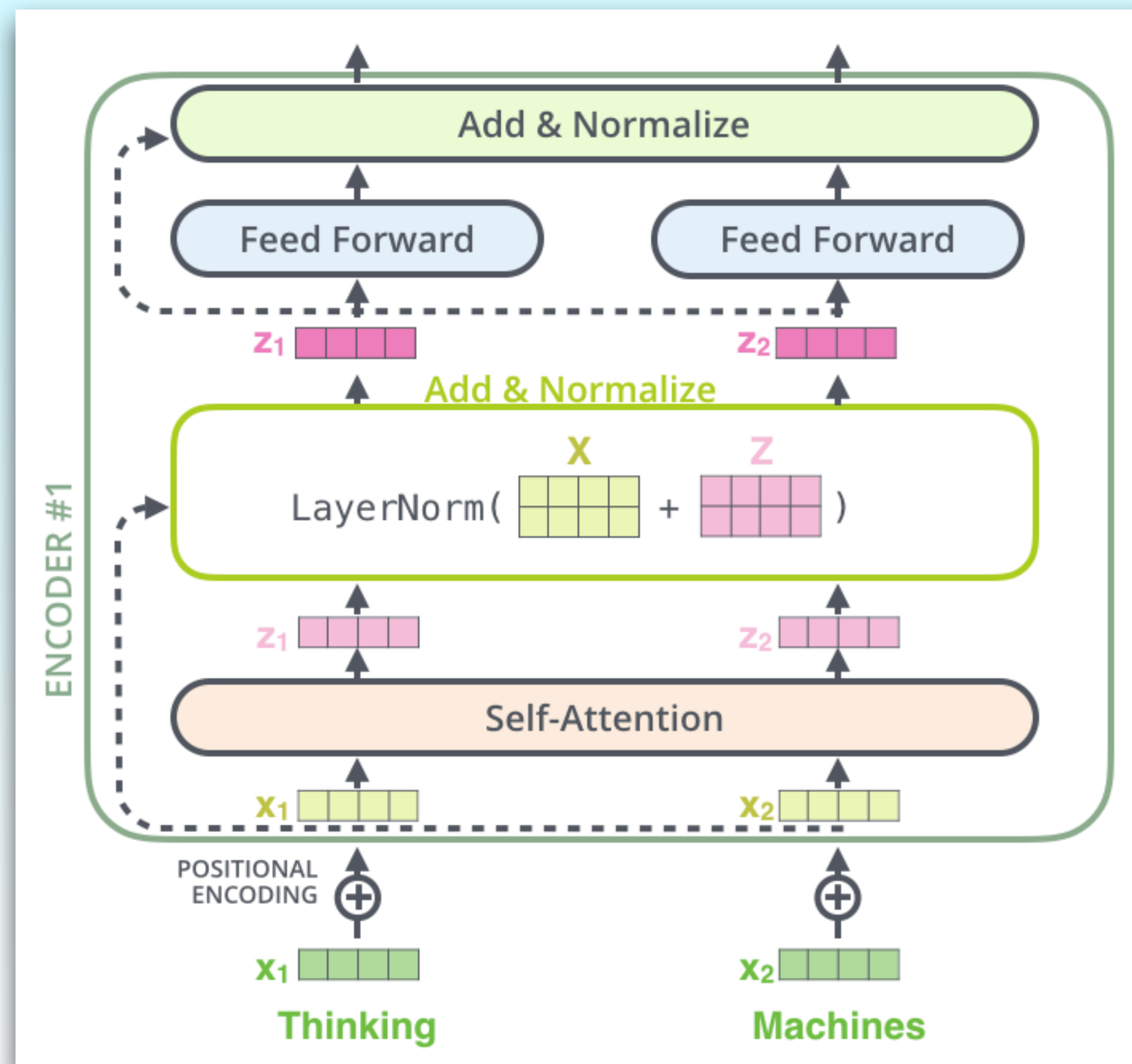

Layer Normalization

What is it?

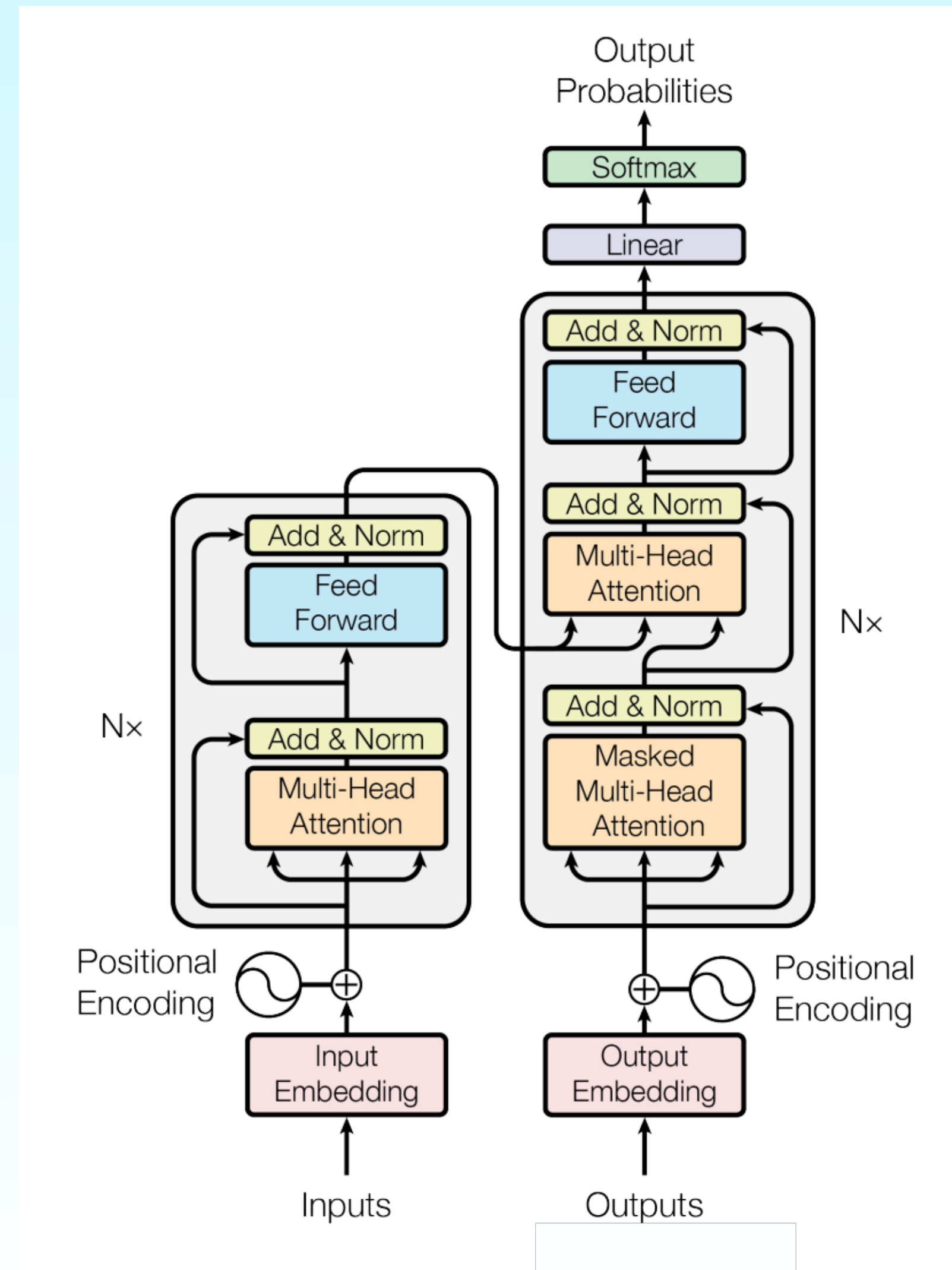
- Normalize the inputs of layers, not batches
- Also used with RNNs
- Common in Transformers
- `torch.nn.LayerNorm`



Full Encoder Together!

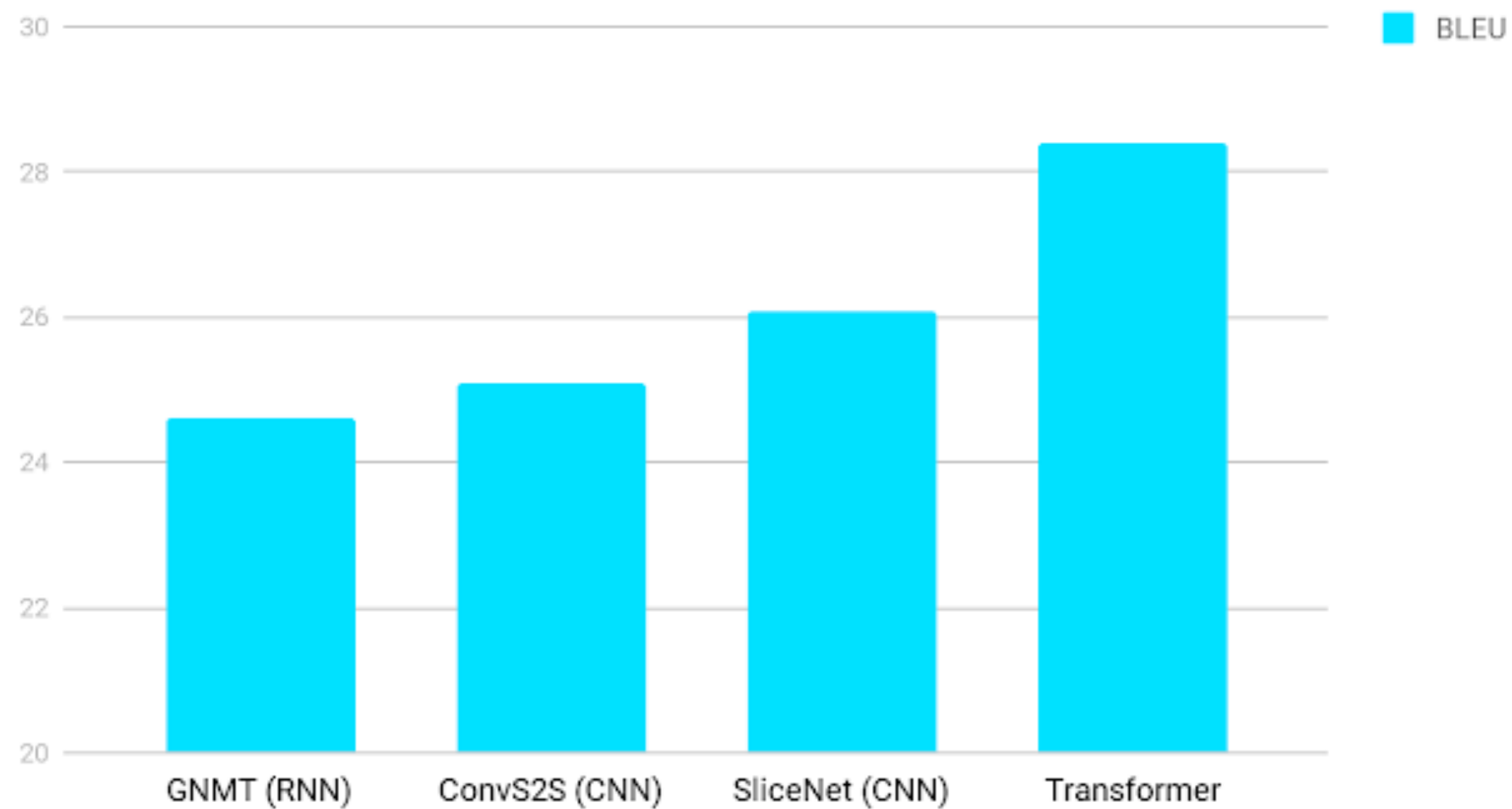


Transformer in one view!

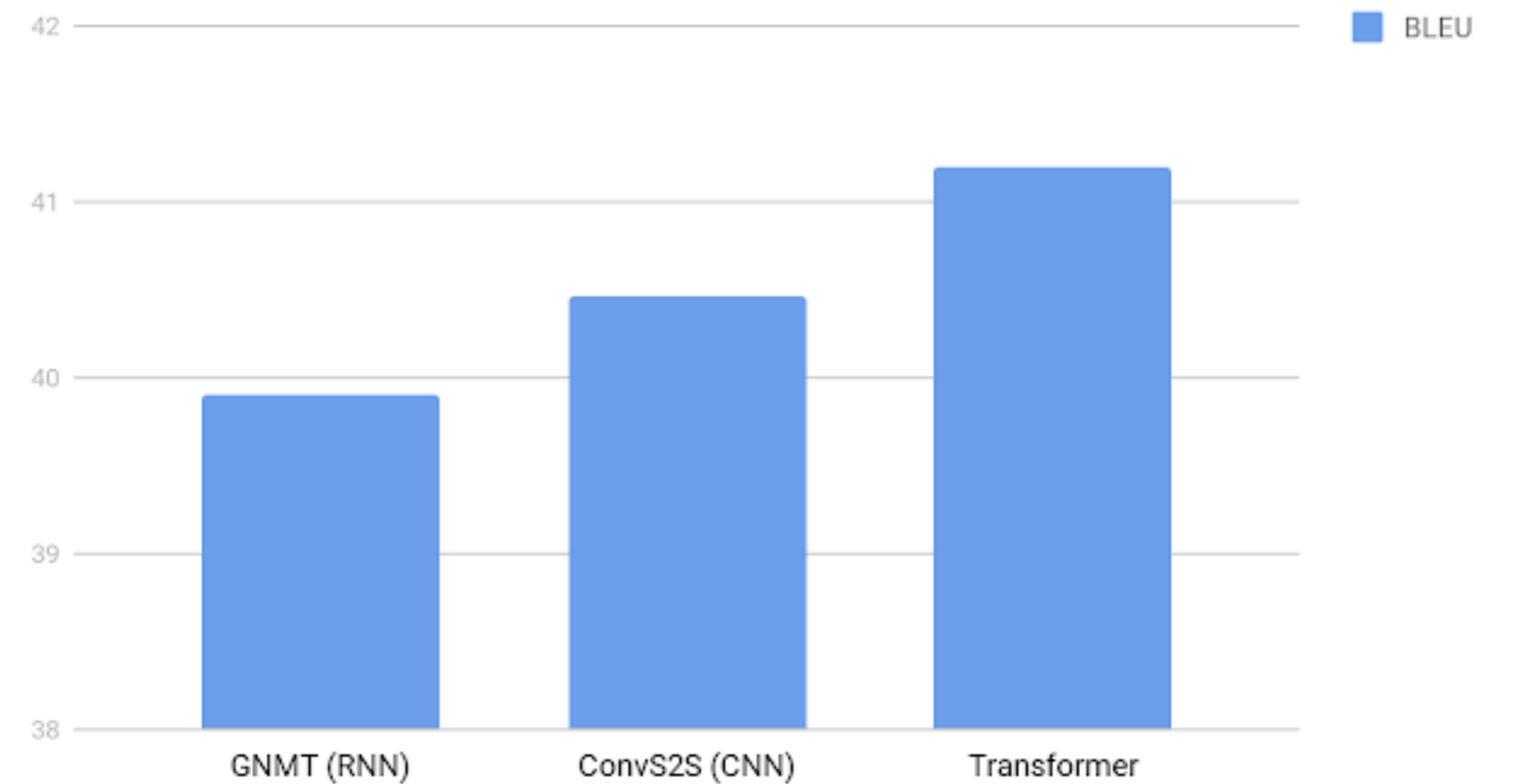


What did they achieve?

English German Translation quality



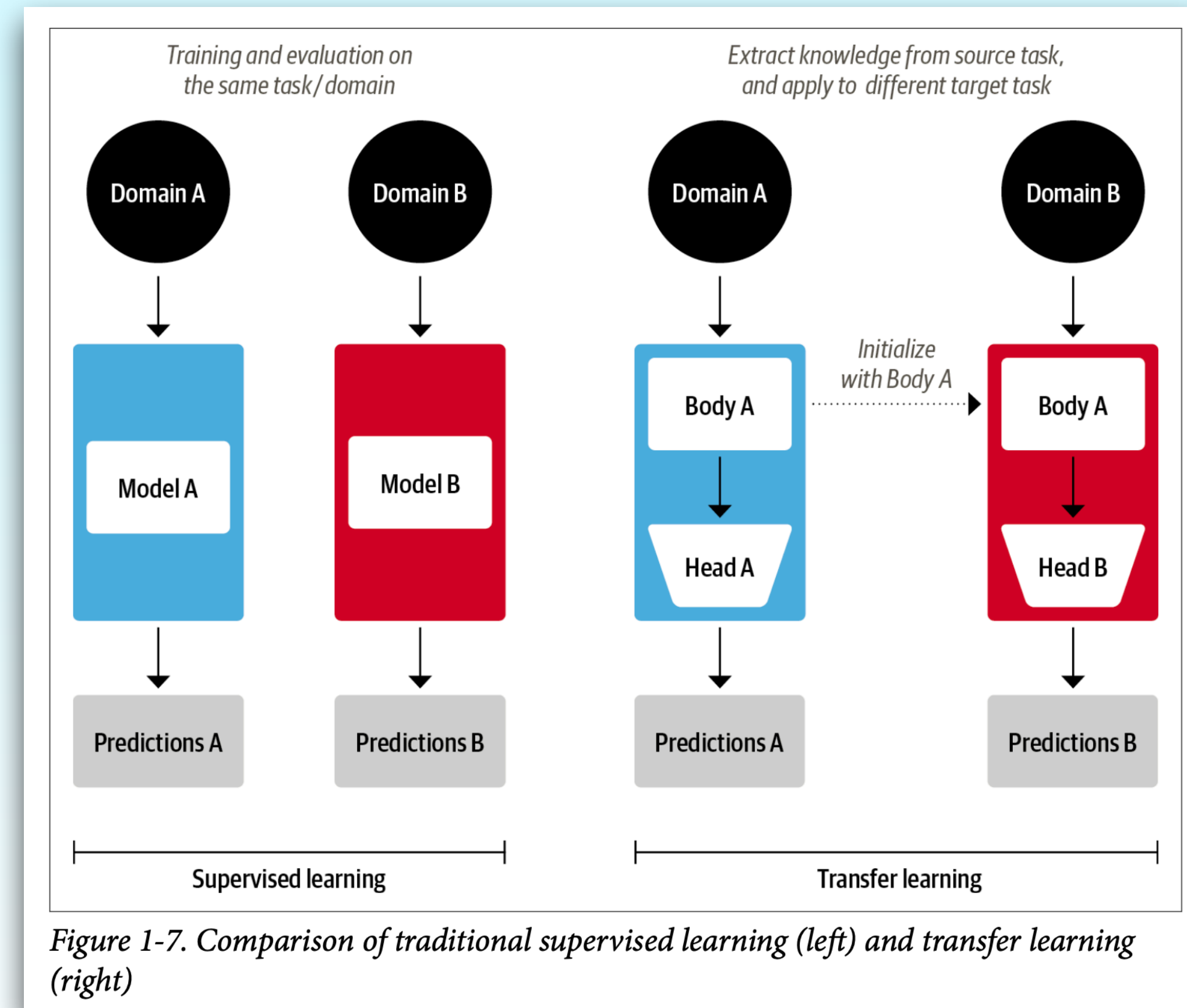
English French Translation Quality



Let's see in code!

Transfer Learning: pre-train + fine-tune

- Model: body + head
 - Body: main weights of the model, constant between tasks; Head: task-specific
- Procedure: train a model on a vast amount of data, then for each specific task, train the head on that task
 - As Tunstall et al. say: “During training, the weights of the body learn broad features of the source domain”
- Allows much less training data to be effective!
- OpenAI was a pioneer around 2017 / 2018 :)



**Let's explore NLP as a specific
deep learning task**

Natural Language Processing

- Any task that works with textual data
- Imagine a model saying if a certain sentence is happy or sad
- Three components:
 - 1) Represent words in a computer-understandable way: **embeddings**
 - 2) The model that gets the words as input
 - 3) Mapping between model outputs and predictions (e.g., the last layer)
- You already know 2 and 3...

Tokenization

- Each sentence contains several words
- Each “token” is a meaningful unit of characters -> discretization!
- “I am attending the summer bootcamp!”
 - I + am + attending + the + summer + bootcamp + !
- *Is this the only way to tokenize the sentence above?*

Why not each word as a token?

- “I am attending the summer bootcamp!”
 - I + am + attending + the + summer + bootcamp + !
- How do you tokenize “check-in” or “six-years-old” or “isn’t”?
- “attend” and “attending” are then totally different tokens, so why not be close?
 - The number of words increases a lot for each variation -> computation
- Some languages do not use space as English does: Korean, Chinese, etc.
- Example alternative:
 - I + am + attend + ing + the + summer + boot + camp + !
- Specific tokens might be needed, e.g., <UNK>

Vocabulary

- Set of all possible tokens
- Any token not in the vocabulary $|V| \rightarrow \langle \text{UNK} \rangle$
- We should represent each word of $|V|$ as a *vector*
- One early solution: one-hot vectors of size $|V|$
 - I $\rightarrow [1, 0, 0, 0, \dots]$
 - am $\rightarrow [0, 1, 0, 0, \dots]$
 - attend $\rightarrow [0, 0, 1, 0, \dots]$
- Downside?

Embeddings

- We want word vectors to contain semantic information
- One-hot does not provide it; how can we find the similarity between these vectors?
 - like -> [0 0 1 0]
 - love -> [0 0 1 0]
 - parameter -> [0 0 1 0]
- Normal time series has semantic information: numbers can be huge, small, or have patterns
 - So, in time series for NLP, the inputs (word vectors) should have meanings
- We should use vectors with real numbers as entries, but how to find them?

Embeddings: learn from the model

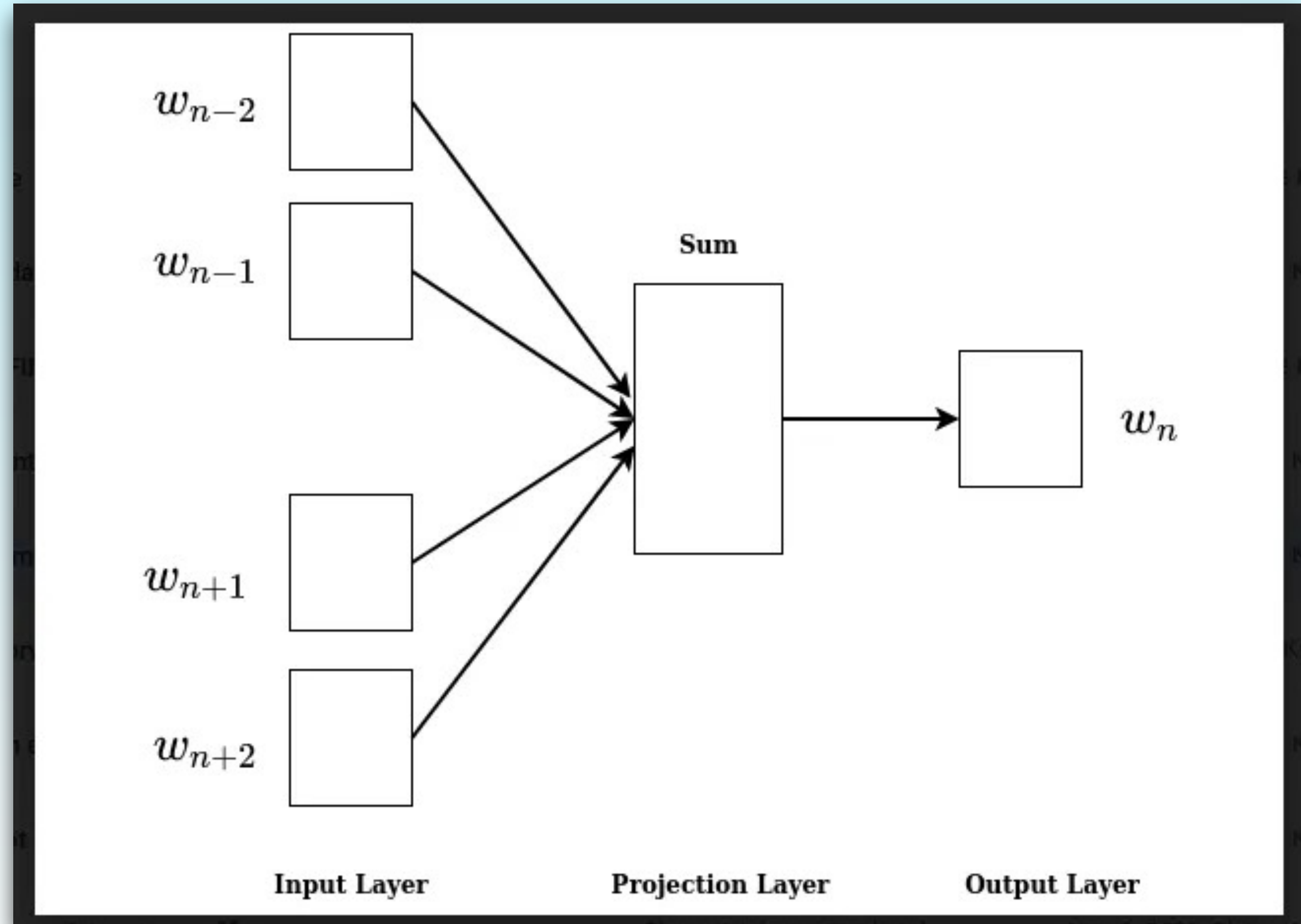
- One possible solution: add feedforward networks, converting one-hot to real-valued vectors
- Is it enough?
- Usually labeled training data is rare, not enough to capture enough dependencies between words
- Idea: use vast amounts of unlabeled text (e.g., on the internet) to find the embeddings, then use those embeddings in our models trained using labeled data
 - Fictional example: use the whole Wikipedia (billions of words) to find embeddings, then use them to train a Transformer or RNN on 1000 sentences to classify sad/happy

Self-supervised Learning of Embeddings

- Find patterns by looking at large bodies of text and words that come next to each other
 - *Imagine a model looking at the whole internet; it should learn that Bern, Zurich, Lausanne, etc. come with the word Switzerland regularly, so they are Swiss cities*
- One pre-transformers approach: word2vec continuous bag of words (CBOW)
 - A model to *predict* a word in a gap, from the surrounding
 - I am the summer bootcamp!
 - attending, enjoying, ...

CBOW

- A certain window size
- When training the model to predict the missing word, it automatically infers the embeddings
- On a big corpus
 - For example: “brown” (1 million words)
- Why not directly training on the targeted data?



Let's see in code!